

Hafta 5 – Kod Sağlamaştırma: Java ve Yorumlanan Diller

Tarih	16.10.2026
Öğrenme çıktıları	ÖÇ.3
Süre	3 saat
Ön bilgi	Java ya da Python'da temel programlama; Hafta 2'den CWE/CVE ve zafiyet sınıflandırma; Hafta 4'ten girdi doğrulama ve güvenli kodlama kuralları; SQL'de basit <code>SELECT</code>
Uygulamalar	<code>code/week-05</code> – 7 demo; Python 3 ve JDK 17+ ister; Windows'ta <code>.\demo.ps1</code> , WSL/Linux'ta <code>sh demo.sh</code>

Bu haftanın sonunda şunları yapabileceksiniz

- Yönetilen dillerin (Java, Kotlin, C#, Python) **hangi hata sınıflarını ortadan kaldırdığını, hangilerini kaldırmadığını** söylemek.
- SEI CERT Oracle Java** kurallarını okumak; girdi doğrulama (`IDS`), seri durumdan çıkarma (`SER`) ve hassas veri (`MSC` , `FIO` , `ERR`) kurallarını kendi kodunuza uygulamak.
- SQL enjeksiyonu, komut enjeksiyonu** ve **yol geçişi** hatalarını göstermek ve parametrelili sorgu, argüman listesi ve kanonikleştirme + kök denetimiyle düzeltmek.
- Güvensiz seri durumdan çıkarmanın** neden tehlikeli olduğunu açıklamak ve `ObjectInputFilter` ile izin listesi kurmak.
- Java bayt kodunun ne kadar okunabilir olduğunu `javap` ile göstermek; **ProGuard / R8** ile küçültme, iyileştirme ve gizleme yapmak, `-keep` kurallarını doğru yazmak.
- Bir projenin **yazılım malzeme listesini (SBOM)** CycloneDX biçiminde üretmek ve bilinen zafiyetli bileşenleri bulmak.

Ders akışı (öğretim üyesi için zaman planı)

Zaman	Bölüm	Ne yapılıyor
0:00–0:15	1–2	Yönetilen diller neyi çözer, neyi çözmez? SEI CERT Oracle Java
0:15–0:50	3–4	Enjeksiyonun ortak kökü; SQL enjeksiyonu — Demo 1
0:50–1:00	Ara	
1:00–1:20	5–6	Komut enjeksiyonu Demo 2 ; yol geçişi Demo 3
1:20–1:50	7–9	Güvensiz seri durumdan çıkarma Demo 7 ; XXE, XSS, şablon; Python ve JavaScript, ReDoS
1:50–2:00	Ara	
2:00–2:15	10	Bayt kodu ve tersine derleme — Demo 4
2:15–2:40	11–13	ProGuard / R8, gelişmiş kurallar; dize gizleme, yansıma Demo 5 ; Android R8, ölçme, statik analiz
2:40–2:55	14	Bağımlılık güvenliği ve SBOM — Demo 6
2:55–3:00	15+	Proje adımı, kendini sına

Laboratuvarı önceden hazırlayın

Bu haftanın demoları **Python 3** ve **JDK 17+** ister. SQL demosunun ana bölümü Python'un yerleşik `sqlite3` modülüyle, hiçbir şey indirmeden çalışır; isteğe bağlı Java bölümleri (JDBC sürücüsü, ProGuard) için her demo klasöründe bir `hazirla` betiği vardır. Betikler yalnız demo klasörüne indirir, sisteme bir şey kurmaz.

```
java -version; python --version
cd cen429-secure-programming\code\week-05\01-sql-enjeksiyonu
.\demo.ps1

sudo apt install -y openjdk-17-jdk python3
cd cen429-secure-programming/code/week-05/01-sql-enjeksiyonu
sh demo.sh
```

⚠ Etik kural — bu derste her hafta geçerli

Enjeksiyon örnekleri yalnız demo klasöründe oluşturulan **sentetik** bir veritabanına ve dosyalara karşı çalışır; enjekte edilen komut yalnız zararsız bir `echo` 'dur. Seri durumdan çıkarma demosu hiçbir saldırı zinciri içermez, yalnız savunmayı gösterir. Bu teknikleri başkasına ait bir web sitesinde, uygulamada ya da veritabanında denemek yasa dışıdır; izinli sızma testleri yazılı bir sözleşme ve kapsam belgesiyle yapılır ([12. hafta](#)).

0. Başlamadan önce

Bu bölüm haftaya hazırlık içindir. Önce bu haftanın dayandığı önceki konuları kısaca hatırlatır; sonra bu haftanın kavramlarını birer cümleyle tanımlayıp her birini ayrıntılı anlatıldığı bölüme bağlar. Önceki haftalarda görülmemiş temel bilgiler ise "Ön bilgi" başlıkları altında sıfırdan anlatılır.

Önceki haftalardan gelenler

- **Bellek hataları (taşma, serbest bellek kullanımı, tanımsız davranış)** — ilk dört hafta boyunca C/C++'ta arabellek taşmasının, serbest bırakılmış belleğin kullanımının ve tanımsız davranışın nasıl oluştuğunu ve önlendiğini gördük ([Hafta 4](#)). Bu hafta §1'de bu hata sınıflarının **hangilerinin** yönetilen dillerde dil düzeyinde ortadan kalktığını, hangilerinin **kalmadığını** karşılaştırıyoruz.
- **SEI CERT kural kitabının yapısı** — dördüncü haftada SEI CERT C/C++ kurallarının kimlik, başlık, hatalı örnek, uyumlu çözüm ve risk bölümlerinden oluşan ortak yapısını görmüştük ([Hafta 4, §2](#)). Bu hafta §2'de aynı yapıyı Java standardında, `IDS / SER / FIO / MSC / ERR` kategorileriyle yeniden kullanıyoruz.
- **TOCTOU (denetim ile kullanım arasındaki zaman)** — ikinci haftada bir dosyanın izin denetimiyle gerçek kullanımı arasındaki zaman farkının bir yarış durumu açığı olduğunu görmüştük ([Hafta 2, §12](#)). Bu hafta §6'da yol geçişine karşı dosya açıldıktan **sonra** da kök içinde kalındığının doğrulanmasını aynı ilkeyle gerekçelendiriyoruz.

Bu haftanın kavram haritası

Kavram	Bir cümlede	Ayrıntısı
Enjeksiyon	Kullanıcıdan gelen verinin bir yorumlayıcı (SQL motoru, kabuk, dosya sistemi, XML ayrıştırıcısı) tarafından veri yerine komut olarak işlenmesidir; bütün enjeksiyon türlerinin ortak kökü budur.	§3
JVM ve bayt kodu	JVM (Java Virtual Machine), Java'yı çalıştıran sanal makinedir; bayt kodu, Java kaynağının derlendiği, makineden bağımsız ara biçimdir ve makine koduna göre çok daha kolay geri çevrilir.	§10
Parametrelili sorgu	Sorgunun metnini sabit tutup kullanıcı verisini ayrı bir kanaldan değer olarak bağlayan SQL çağrısıdır (prepared statement); veri hiçbir zaman sorgunun yapısını değiştiremez.	§4
Seri durumdan çıkarma	Bir bayt dizisinden nesne oluşturmaktır (deserialization); akış güvenilmezse hangi sınıfın oluşturulacağını saldırgan seçebilir.	§7
XXE (XML dış varlık)	Bir XML ayrıştırıcısının, belgedeki <code><!DOCTYPE></code> /varlık tanımlarını kullanarak yerel dosya ya da ağ adresi okumasının kötüye kullanılmasıdır.	§8
Yol geçişi	<code>../</code> gibi dizilerle izin verilen bir klasörün dışına çıkıp kök dizinin dışındaki dosyalara erişmektir (path traversal).	§6
ProGuard / R8	Java bayt kodunu küçülten, iyileştiren ve sınıf/metot adlarını anlamsız adlarla değiştirerek gizleyen araçlardır; R8, ProGuard'ın Android'deki halefidir.	§11
-keep kuralı	ProGuard/R8'e belirtilen sınıf ya da metodun adını değiştirmemesini söyleyen yapılandırma kuralıdır; yansımayla adıyla çağrılan kod bu kural olmadan çalışma anında bulunamaz.	§11
SBOM	Bir yazılımın içindeki bütün bileşenlerin makine tarafından okunabilir listesidir (Software Bill of Materials); bir bağımlılıkta açık çıktığında "etkilendik mi?" sorusunu hızlı yanıtlar.	§14

Ön bilgi: Yönetilen diller ve çöp toplama

Bir **yönetilen dil** (managed language — Java, C#, Python, JavaScript gibi), belleği programcı yerine çalışma ortamının otomatik yönettiği bir dildir; programcı `malloc / free` çağırmasın, bunun yerine bir **çöp toplayıcı** (garbage collector, GC) artık kullanılmayan nesneleri arka planda otomatik olarak temizler. Bu tek fark, ilk dört haftada C/C++'ta gördüğümüz taşma ve serbest bellek kullanımı gibi hataların büyük kısmının dil düzeyinde ortadan kalkmasını sağlar; §1 bunu ayrıntılı biçimde gösterir.

Ön bilgi: Veritabanları ve SQL

SQL, bir veritabanına sorgu göndermek için kullanılan sorgulama dilidir (`SELECT ... WHERE ...` gibi); bir uygulama, kullanıcıdan aldığı bir değeri (kullanıcı adı, arama terimi) bu sorgunun içine koyarak veritabanından veri okur ya da yazar. Bu değer sorgunun metnine kaçışsız biçimde eklenirse §3–4'te göreceğimiz SQL enjeksiyonu ortaya çıkar.

Ön bilgi: Yansıma ve bağımlılıklar

Yansıma (reflection), bir sınıfı ya da metodu kaynak kodda doğrudan adıyla yazmak yerine **çalışma anında** bir dizeden (`Class.forName("...")`, `getMethod("...")` gibi) bulup çağırma yeteneğidir; §12 bunun gizlemeyle nasıl etkileştiğini gösterecek. **Bağımlılık** (dependency), projenizin kullandığı, kendi ekibinizin yazmadığı dış bir kütüphanedir; kendi kodunuz güvenli olsa bile, kullandığınız bir bağımlılıktaki bir açık uygulamanızı da etkiler — §14 bunu SBOM ile ele alır.

Kavramlar birbirine nasıl bağlanır?

Yukarıdaki terimler rastgele bir liste değildir; her biri bir öncekinin üstüne oturur. Aşağıdaki tablo "bunu anlamak için önce şunu bilmem gerekir" ilişkisini gösterir — bir terimi unutursanız tablodaki "önkoşul" sütununa geri dönün.

Terim	Önkoşulu	Hangi bölümde derinleşir?
Enjeksiyon	Yorumlayıcı, veri/komut ayrımı	Bölüm 3
SQL enjeksiyonu / parametrelili sorgu	Enjeksiyon, SQL sözdizimi	Bölüm 4
Komut enjeksiyonu	Enjeksiyon, kabuk kavramı	Bölüm 5
Yol geçişi	Enjeksiyon, dosya sistemi yolu	Bölüm 6
Seri durumdan çıkarma	Nesne, sınıf, bayt dizisi	Bölüm 7
XXE / XSS / şablon enjeksiyonu	Enjeksiyon, XML/HTML	Bölüm 8
ReDoS	Düzenli ifade, geri izleme	Bölüm 9
Bayt kodu	JVM	Bölüm 10
ProGuard/R8, <code>-keep</code>	Bayt kodu, reflection	Bölüm 11–13
SBOM	Bağımlılık	Bölüm 14

Bu tablonun okunuş yönü önemlidir: **satır başındaki terimi anlamak için önce "önkoşulu" sütunundaki terimi bilmelisiniz**. Örneğin "parametrelili sorgu" bölümünü okumadan önce "enjeksiyon" kavramının ("veri, komutla aynı kanaldan geçerse komut sanılır") oturmuş olması gerekir; aksi hâlde "neden ? işareti güvenli de dize birleştirme değil?" sorusu havada kalır.



Bu haftayı nasıl okumalı?

Bölümler **sırayla** birbirinin üstüne inşa edilir: 3 (enjeksiyonun ortak kökü) → 4–8 (her enjeksiyon türü aynı kökten türer) → 9 (aynı kök, farklı diller) → 10–13 (bayt kodu ve gizleme, ayrı bir konu: koda gömülü sırlar) → 14 (bağımlılık güvenliği, üçüncü bir konu). Bölüm 3'ü atlayıp Bölüm 4'e geçerseniz, "aynı hatanın farklı yüzü" cümleleri anlamsız kalır; zor gelen bir bölümü ikinci kez okuyup ilerleyin.

1. Yönetilen diller neyi çözer, neyi çözmez?

İlk dört hafta C ve C++ ile geçti: arabellek taşması, serbest bırakılmış bellek, tamsayı taşması, tanımsız davranış. Java, Kotlin, C#, Python, JavaScript gibi **yönetilen** (managed) dillerde bu hataların büyük kısmı **dil düzeyinde** ortadan kalkar:

Yönetilen dil neyi çözer, neyi çözmez?

Java/Kotlin/C# çöp toplayıcı + sınır denetimi

ÇÖZER (dil düzeyinde)

tampon taşması
use-after-free
çift serbest bırakma
ilklenirilmemiş değişken
→ bellek güvenliği

ÇÖZMEZ

enjeksiyon (SQL, komut, XML)
güvensiz deserialization
koda gömülü sırlar
zafiyetli bağımlılıklar
mantık/yetki hataları

Dil bir hata SINIFINI kaldırır; kalan sınıflar için hâlâ siz sorumlusunuz.

🔗 Kısa tarihçe: yönetilen diller ve kalan açıklar

- **1995** — Java ve JVM: çöp toplayıcı ve sınır denetimi **bellek hatalarının** çoğunu dil düzeyinde kaldırır.
- **1998** — SQL enjeksiyonu ilk kez belgelenir (Rain Forest Puppy); **2003 OWASP Top 10** yayımlanır — bellek değil **mantık/enjeksiyon** hataları öne çıkar.
- **2002** — ProGuard (Java küçültme/gizleme), sonra **R8**: bayt kodun kolay geri çevrilmesine yanıt.
- **2015** — Java **deserialization** saldırıları; **2020–21 SolarWinds** ve **Log4Shell** tedarik zinciri/**SBOM** çağını başlatır.

Ana fikir: dil bellek hatasını çözer, **enjeksiyonu ve bağımlılık riskini çözmez**.

Hata sınıfı	C/C++	Java / yönetilen diller
Arabellek taşması	Programcının sorumluluğu	Her dizi erişimi denetlenir → <code>ArrayIndexOutOfBoundsException</code>
Serbest bırakılmış bellek, çift serbest bırakma	Programcının sorumluluğu	Çöp toplayıcı: bellek, kimse göstermediğinde serbest bırakılır
İklendirilmemiş değişken	Tanımsız davranış	Derleyici hata verir ya da alan varsayılan değerle başlar
İşaretili tamsayı taşması	Tanımsız davranış	Tanımlı: sarar (ama mantık hatası sürer)
Biçim dizesi (%n)	Belleğe yazma	<code>String.format</code> belleğe yazmaz (ama günlük enjeksiyonu sürer)

Bu büyük bir kazançtır; ama "Java kullanıyoruz, güvendeyiz" demek için yeterli değildir. Aşağıdaki sınıflar dilden bağımsızdır ve web uygulamalarından mobil uygulamalara kadar en çok bulunan zafiyetlerdir:

Hata sınıfı	Neden dil çözmez?	Bu hafta
Enjeksiyon (SQL, komut, yol, XML, şablon)	Veri ile komutun karışması bir tasarım hatasıdır	Demo 1–3
Güvensiz seri durumdan çıkarma	Dilin kendi özelliği kötüye kullanılır	Demo 7
Koda gömülü sırlar	Bayt kodu kolayca okunur	Demo 4
Tersine mühendislik	Bayt kodu yüksek düzeylidir; adlar ve dizeler korunur	Demo 4–5
Zafiyetli bağımlılıklar	Uygulamanın büyük kısmı başkasının kodudur	Demo 6
Mantık ve yetkilendirme hataları	Dilin bilemeyeceği iş kurallarıdır	2. hafta

İşlenmiş örnek: bir dizi taşması Java'da neden çöker de C'de çökmez

Dördüncü haftada `buf[10]` gibi 10 elemanlı bir diziye 10. indeksle (11. eleman) yazmanın C'de **tanımsız davranış** olduğunu, çoğu zaman hiçbir hata vermeden bitişik belleği bozduğunu görmüştük. Aynı hatayı Java'da adım adım izleyelim.

DiziTasmasi.java

```
public class DiziTasmasi {
    public static void main(String[] args) {
        int[] dizi = new int[10]; // 10 elemanlık dizi: indeks 0..9 geçerli
        dizi[9] = 42; // geçerli: son eleman
        System.out.println("dizi[9] = " + dizi[9]);
        dizi[10] = 99; // GECERSİZ: 11. eleman yok
    }
}
```

javac DiziTasmasi.java && java DiziTasmasi

```
dizi[9] = 42
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException:
    Index 10 out of bounds for length 10
    at DiziTasmasi.main(DiziTasmasi.java:6)
```

Adım adım ne oldu:

- JVM her dizi erişiminde (`dizi[10]`) **önce** indeksin $0 \leq i < \text{uzunluk}$ sınırında olup olmadığını denetler; bu denetim derleyicinin ürettiği bayt kodunun **her** `aload / iastore` **öncesinde** çalışan bir parçasıdır.
- `10`, dizinin uzunluğu olan `10`'a eşit ya da büyük olduğu için denetim başarısız olur.
- JVM bitişik belleğe **yazmaz**; bunun yerine `ArrayIndexOutOfBoundsException` fırlatır ve programı (ya da `try / catch` varsa yalnız o bloğu) kontrollü biçimde durdurur.
- Hangi satırda, hangi indeksle olduğu **hata iletisinde** görünür — dördüncü haftadaki C örneğinde ise sessiz bir bellek bozulması ya da rastgele bir çökme olurdu, hatanın kaynağını bulmak saatler sürebilirdi.

✓ Kural

Java'da bir dizi taşması **her zaman** bir istisna olarak yakalanır; programın çökmesi ya da hatalı davranması değil, **kontrollü** durması beklenir. Bu, "yönetilen dil bellek güvenliğini garanti eder" cümlesinin somut karşılığıdır — ama bu garantinin enjeksiyon, seri durumdan çıkarma ya da mantık hataları için **geçerli olmadığını** unutmayın; onlar bu haftanın geri kalanının konusu.

🔗 Yönetilen dil içinde native kod

Bir Java uygulaması bellek güvenliğini ancak **tamamen** Java'da kaldığı sürece kazanır. Performans ya da güvenlik için JNI ile çağrılan C/C++ kodu ([1. haftadaki](#) mobil ödeme mimarisindeki native katman gibi), ilk dört haftanın bütün riskini yeniden taşır. Sahada bu yüzden iki taraf ayrı ayrı sağlamlaştırılır: native taraf 4. haftanın yöntemleriyle, Java tarafı bu haftanın yöntemleriyle; iki taraf da birbirinin bütünlüğünü denetler.

2. SEI CERT Oracle Java: yönetilen dilin kural kitabı

[Dördüncü haftada](#) tanıdığımız SEI CERT'in Java standardı da aynı yapıdadır: kimlik, başlık, hatalı örnek, uyumlu çözüm, risk. Kimlikler `-J` sonekiyle biter.

SEI CERT Oracle Java: aynı yapı, farklı dil

yönetilen dil bellek hatasını çözer, mantık hatasını çözmez



IDS (girdi doğrulama), SER (serileştirme), FIO (dosya), MSC (çeşitli) — enjeksiyon konuları IDS'tedir.

Kısaltma	Kategori	Bu haftaya dokunan kural
IDS	Girdi doğrulama ve veri temizleme	IDS00-J: güvenilmeyen verinin SQL enjeksiyonunu önle · IDS07-J: güvenilmeyen veriyi <code>Runtime.exec()</code> 'e verme · IDS16-J: XML enjeksiyonunu önle · IDS17-J: XML dış varlık saldırılarını önle
FIO	Girdi/çıkı	FIO16-J: yol adlarını doğrulamadan önce kanonikleştir
SER	Serileştirme	SER12-J: güvenilmeyen verinin seri durumdan çıkarılmasını önle
MSC	Çeşitli	MSC03-J: koda gömülü hassas bilgi kullanma
ERR	Hata işleme	ERR01-J: istisnalarda hassas bilgiyi sızdırma
FIO	Günlük	FIO13-J: hassas bilgiyi güven sınırının dışına günlüğe yazma
OBJ	Nesne yönelimi	OBJ01-J: alanların erişilebilirliğini sınırla
SEC	Platform güvenliği	SEC00-J: ayrıcalıklı blokların hassas bilgiyi sızdırmasına izin verme

Neden CERT kuralları hâlâ gerekli, Java bellek hatalarını çözdüyse?

Dördüncü haftada C için gördüğümüz SEI CERT C kuralları (`ARR` , `STR` , `MEM` ...) büyük ölçüde bellek yönetimi hatalarını hedefliyordu; JVM bu sınıfı otomatik olarak kapattığı için Java standardında `MEM` kategorisi neredeyse boş. Ama tablodaki `IDS` , `SER` , `FIO` , `MSC` , `ERR` kategorileri **dilin çözmediği** hatalardır: girdi doğrulama, seri durumdan çıkarma, hassas bilginin sızması gibi konular bellekle değil **tasarım kararlarıyla** ilgilidir. Bu yüzden CERT, dilden dile "hangi kategori büyür, hangisi küçülür" ilkesiyle çalışır: C standardında `MEM` ve `ARR` kalın, Java standardında `IDS` ve `SER` kalın basılıdır.



OWASP ile birlikte okumak

Web uygulamaları için **OWASP Top 10** (2. hafta) ve ASVS, mobil uygulamalar için **OWASP MASVS** (özellikle MASVS-CODE ve MASVS-RESILIENCE) aynı konuları uygulama türüne göre sıralar. CERT kuralları "kodu nasıl yazarım?", OWASP belgeleri "neyi test ederim?" sorusunu cevaplar.

3. Enjeksiyonun ortak kökü: veri ile komutun karışması (Tarif 3.11)

SQL enjeksiyonu, komut enjeksiyonu, yol geçişi, XML enjeksiyonu, şablon enjeksiyonu, 2. haftadaki günlük enjeksiyonu ve 4. haftadaki biçim dizesi açığı aynı hatanın farklı yüzleridir: bir **yorumlayıcıya** (SQL motoru, kabuk, dosya sistemi, XML ayrıştırıcı, `printf`) verilen metnin içinde **komut** ile **veri** aynı kanaldan geçer. Kullanıcı verisi, yorumlayıcının özel karakterlerini (`'` , `;` , `..` / , `<` , `%`) içeriyorsa, veri komutun bir parçası olur.

Enjeksiyonun ortak kökü: veri ile komutun karışması

bütün enjeksiyon türleri aynı şemanın farklı yorumlayıcılardaki hâli

HATALI: tek kanal

"...WHERE x='" + girdi

Yorumlayıcı girdiyi
KOMUT olarak ayrıştırır

SQLi · komut enj. · XXE
şablon enjeksiyonu

DOĞRU: ayrı kanal

WHERE x = ? (parametre)

Komut SABİT kalır,
girdi yalnız VERİ

enjeksiyon ailesi
tümüyle kapanır

Kaçış (escaping) son çaredir; asıl çözüm komutu ve veriyi ayrı kanala koymaktır.

Bu yüzden bütün enjeksiyonların **asıl** düzeltilmesi aynıdır: **komut ile veriyi ayrı kanallardan ver**. Kaçış karakteriyle temizlemek (' → ') ikinci savunma hattıdır; her yorumlayıcının kaçış kuralları farklıdır ve bir tanesini unutmak yeter.

Yorumlayıcı	Tek kanal (hatalı)	İki kanal (doğru)
SQL motoru	"SELECT ... WHERE ad='" + ad + "'"	PreparedStatement + ? parametreleri
Kabuk	Runtime.exec("sh -c 'ping " + host + "'")	ProcessBuilder("ping", host) — kabuk yok
Dosya sistemi	new File(kok, istek)	Kanonikleştir + kök içinde mi denetle
XML	Dize birleştirerek XML kurmak	DOM / StAX API'si ile öge ve öznitelik oluşturmak
HTML	" " + yorum + " "	Şablon motorunun otomatik kaçışı, bağlama göre kodlama
printf	printf(girdi)	printf("%s", girdi)

Yorumlayıcı girdiyi neden "kod" sanır? Adım adım

Öğrencilerin en çok kaçırdığı halka budur: **yorumlayıcı, kendisine gelen dizinin hangi karakterlerinin "programcının yazdığı sabit kod", hangilerinin "kullanıcının girdiği veri" olduğunu bilmez**. Elindeki tek şey, art arda gelen karakterlerdir. Bunu somutlaştıralım; "SELECT ... WHERE ad = '" + ad + "'" ifadesinin çalışma zamanında oluşturduğu dizeyi bir SQL motorunun nasıl okuduğunu karakter karakter izleyelim (ad değişkeninin içinde x' OR '1'='1 girdisi olduğunu varsayalım):

1. Java önce dize birleştirmeyi yapar; sonuçta ortaya **tek bir düz metin** çıkar: SELECT ... WHERE ad = 'x' OR '1'='1'. Bu noktadan itibaren "bu parça geliştiricinin yazdığı", "bu parça kullanıcının girdiği" bilgisi **kaybolmuştur** — elde yalnız karakterler vardır.
2. Bu düz metin SQL motoruna **tek bir komut** olarak gönderilir. Motorun ayrıştırıcısı (lexer/parser) dizeyi soldan sağa, karakter karakter tarar ve her karaktere göre bir **duruma** girer.

3. `WHERE ad =` okunduktan sonra bir boşluk, sonra bir `'` karakteriyle karşılaşır → ayrıştırıcı "dize sabiti başladı" durumuna geçer. Bundan sonraki her karakteri (harf, boşluk, `0`, `R`...) bu dize sabitinin **içeriği** sayar, bir sonraki `'` karakterine kadar.
4. Girdideki ilk `'` (kullanıcının yazdığı) tam olarak bu bekleneni sağlar: ayrıştırıcı "dize sabiti bitti" der ve **normal SQL söz dizimine geri döner**.
5. Geri dönülen noktada ayrıştırıcının önünde artık `OR '1'='1'` metni durur; bu metin **SQL anahtar sözcükleri ve operatörleri** olarak okunur (`OR`, `=`), çünkü ayrıştırıcı "dize sabiti" durumundan çıkmıştır.
6. Sonuç: motor tek bir sorgu ayrıştırmıştır, ama bu sorgunun **mantıksal yapısı** (`... AND ...` yerine `... OR her_zaman_dogru`) kullanıcının girdiği karakterler tarafından belirlenmiştir.

Kritik nokta şudur: ayrıştırıcı **hata yapmıyor**; tam olarak SQL söz dizimi kurallarına göre davranıyor. Hata, yorumlayıcıya "kodun bir parçası mısın, veri misin?" bilgisini taşıyan **ayrı bir kanal olmamasıdır**. Aynı adım dizisi (bir yorumlayıcı → karakter karakter tarama → özel karakterde durum değişikliği) kabuk için `;`, dosya sistemi için `../`, XML için `</ &` ile birebir tekrarlanır; yalnız "özel karakter" ve "durum makinesi" değişir. Bölüm 4–8, bu adımların her yorumlayıcıdaki somut karşılığını gösterecek.



Kitaptaki karşılığı

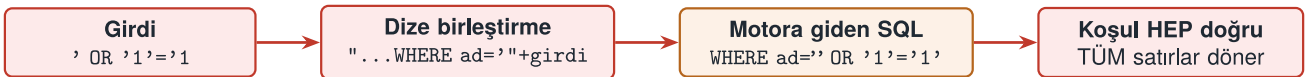
Viega ve Messier, Tarif 3.11'de SQL enjeksiyonunu, 3.10'da siteler arası betiği (XSS), 3.7'de dosya adı ve yol doğrulamayı ve 1.7'de dış programları güvenli çalıştırmayı işler. Kitap C üzerinden anlatır ama ilkeler dilden bağımsızdır: **parametrelili arayüz kullan, kabuğu atla, kanonikleştir, beyaz liste uygula**.

4. SQL enjeksiyonu

Bir giriş formunun arkasında şöyle bir kod olduğunu düşünün:

SQL enjeksiyonu adım adım

girdi tırnağı kapatıp kendi mantığını ekliyor



Parametrelili sorguda motor komutu ÖNCE derler; girdi artık mantık değiştiremez.

Hatalı: sorgu dize birleştirmeyele kuruyor

```
String sql = "SELECT id, rol FROM kullanicilar WHERE ad = '" + ad +
    "' AND parola_ozeti = '" + ozet + "'";
ResultSet rs = baglanti.createStatement().executeQuery(sql);
```

Kullanıcı adı alanına `ayse` yazıldığında sorgu beklendiği gibidir. Ama kullanıcı `ayse' --` yazarsa sorgu şuna dönüşür:

```
SELECT id, rol FROM kullanicilar WHERE ad = 'ayse' --' AND parola_ozeti = '...'
```

-- SQL'de satır sonuna kadar yorum demektir: parola denetimi sorgudan **silinmiştir**. Girdideki tek bir tırnak işareti, sorgunun **yapısını** değiştirdi. Aynı teknikle saldırgan başka tabloları okuyabilir (`UNION SELECT`), kayıtları değiştirebilir ya da silebilir; veritabanı yapılandırmasına göre işletim sistemi komutu bile çalıştırabilir. SQL enjeksiyonu yirmi yılı aşkın süredir OWASP Top 10 listelerinin üst sıralarındadır (CWE-89).

Doğrusu: parametrelili sorgu

Doğru: PreparedStatement

```
String sql = "SELECT id, rol FROM kullanicilar WHERE ad = ? AND parola_ozeti = ?";
try (PreparedStatement ps = baglanti.prepareStatement(sql)) {
    ps.setString(1, ad);
    ps.setString(2, ozet);
    try (ResultSet rs = ps.executeQuery()) {
        /* ... */
    }
}
```

Sorgunun **metni** sabittir ve veritabanına önce gönderilip derlenir; değerler ayrı bir kanaldan gelir. Değer ne içerirse içersin (`'`, `--`, `;`), yalnız bir **değer** olarak karşılaştırılır; sorgunun yapısına dokunamaz. Bu, önceki bölümdeki "iki kanal" ilkesinin tam karşılığıdır (IDS00-J).

Parametrelili sorgu mekanik olarak neden çalışır?

Bölüm 3'te ayrıştırıcının karakter karakter çalıştığını gördük. `PreparedStatement` bu mekanizmayı **iki aşamaya** böler; sihir burada gizlidir:

- Hazırlama (prepare) aşaması:** `baglanti.prepareStatement(sql)` çağrıldığında, veritabanı sunucusuna yalnız `SELECT id, rol FROM kullanicilar WHERE ad = ? AND parola_ozeti = ?` metni gönderilir — kullanıcının girdisi **henüz ortada yoktur**. Sunucu bu metni ayrıştırır (Bölüm 3'teki durum makinesiyle), bir **sorgu planı** (query plan) üretir ve bu planı bellekte tutar. `?` işaretleri plan içinde "buraya bir değer gelecek" diye işaretlenmiş **yer tutuculardır**.
- Bağlama (bind) aşaması:** `ps.setString(1, ad)` çağrıldığında, `ad` değişkeninin içeriği **hiçbir zaman SQL metniyle birleştirilmez**. Sürücü, değeri ayrı bir ikili protokol mesajıyla (JDBC/veritabanı arasındaki tel protokolü) "1 numaralı yer tutucunun değeri şu bayt dizisidir" diye gönderir.
- `ps.executeQuery()` çalıştığında veritabanı, **zaten ayrıştırılmış olan** plana, gelen değerleri doğrudan yerleştirir. Değerin içinde `'` ya da `--` olsa bile, bu karakterler **hiçbir zaman ayrıştırıcıdan geçmez** — ayrıştırma çoktan bitmiştir. Karakterler yalnız "bu sütunla karşılaştırılacak bayt dizisi" olarak kopyalanır.

Bunu Bölüm 3'teki `printf("%s", girdi)` örneğiyle karşılaştırmak öğretici: `%s` 'in yerine konan dize, biçim dizesindeki bir `%n` gibi **yeniden yorumlanmaz**; olduğu gibi yazdırılır. Parametrelili sorguda da `?` 'nin yerine konan değer, sorgu metni gibi **yeniden ayrıştırılmaz**; olduğu gibi karşılaştırılır. İki mekanizma da aynı ilkeye dayanır: **yapıyı belirleyen ayrıştırma bir kez, veri bağlama ayrı ve sonra**.



Sık yapılan hata: yalnız tek tırnağı kaçışlamak

Bazı geliştiriciler `ad.replace("'", "''")` gibi bir satırla "SQL enjeksiyonunu çözdüm" sanır. Sonuç: sayısal bir alanda (`WHERE yas = " + yas`) tek tırnak hiç yoktur, kaçışlama hiçbir şey yapmaz; `LIKE` deseninde `%` farklı bir özel karakterdir; MySQL, PostgreSQL, Oracle'ın kaçış kuralları birbirinden farklıdır; bazı sürücüler Unicode ya da çok baytlı kodlamalarda kaçışlamayı atlatan diziler kabul eder. Tek bir unutulmuş karakter sınıfı, bütün savunmayı geçersiz kılar.

✓ Kural

Kaçışlama SQL enjeksiyonu için **birincil** savunma değildir; en fazla ek bir güvenlik katmanıdır. Birincil savunma her zaman parametrelili sorgu (ya da ORM'in parametre bağlama arayüzü) olmalıdır — çünkü kaçışlama "özel karakterleri tanıyıp temizlemeye" dayanırken, parametrelili sorgu özel karakterlerin **hiç ayırıştırıcıya uğramamasını** sağlar.

Parametrelili sorgunun yetmediği yerler

Durum	Neden?	Çözüm
Tablo ya da sütun adı (ORDER BY sütunu)	? yalnız değer yerine konabilir, ad yerine konamaz	Beyaz liste: <code>Map<String,String> izinli = {"ad"→"ad", "tarih"→"kayıt_tarihi"}</code>
LIKE desenleri	% ve _ joker karakterlerdir	Değeri parametre yapın, jokerleri ayrıca kaçışlayın
ORM içinde elle yazılmış sorgu	JPQL / HQL de bir sorgu dilidir; birleştirme aynı hatayı yapar	ORM'in parametre bağlama arayüzü (<code>setParameter</code>)
Saklı yordam içinde dinamik SQL	Yordamın içinde birleştirme yapılıyorsa sorun taşınmıştır	Yordamın içinde de parametre kullanın

Derinlemesine savunma

- **En az ayrıcalık:** Uygulamanın veritabanı kullanıcısı yalnız gerektiği tablolara, yalnız gerektiği işlemlerle erişsin. Giriş ekranının kullanıcısının `DROP TABLE` yetkisi olmamalı.
- **Hata iletisi:** Veritabanı hatasını kullanıcıya olduğu gibi göstermek, sorgunun yapısını ve tablo adlarını sızdırır (ERR01-J). Kullanıcıya genel bir ileti, ayrıntı güvenli günlüğe.
- **Girdi doğrulama:** Beklenen biçim bilinen alanlarda (kimlik numarası, tarih, e-posta) beyaz liste doğrulaması saldırı yüzeyini daraltır; ama parametrelili sorgunun **yerine geçmez**.

İşlenmiş örnek: Demo 1'in çıktısını satır satır okumak

Demo 1'in Python bölümü (`sql.py`) üç sentetik kullanıcı (`ayse` , `mehmet` , yönetici `admin`) içeren bir SQLite veritabanı kurar ve aynı giriş sorgusunu iki yolla çalıştırır. Kodu okuyup çalıştırdığınızda göreceğiniz çıktıyı adım adım izleyelim; her adımda **neden** o sonucun çıktığını Bölüm 3–4'teki mekanizmaya bağlıyoruz.

ADIM 1 — Dürüst giriş: `ad='ayse' parola='parola123'`

```
[KOTU YOL]
Uretilen SQL:
SELECT id, ad, rol FROM kullanıcı WHERE ad = 'ayse' AND parola = 'parola123'
-> 1 satır dondu. GIRIS BASARILI:
    id=1 ad=ayse rol=kullanici
[IYI YOL]
Uretilen SQL (sablon):
SELECT id, ad, rol FROM kullanıcı WHERE ad = ? AND parola = ?    [degerler ayrica gonderilir]
-> 1 satır dondu. GIRIS BASARILI:
    id=1 ad=ayse rol=kullanici
```

Beklendiği gibi: girdi zararsız olduğunda hatalı kod da doğru kod da aynı sonucu üretir. Bu yüzden hatalı kod geliştirme sırasında **fark edilmez** — testler dürüst girdiyle yapılır.

ADIM 2 — SALDIRI: parola alanına ' OR '1'='1 yaziliyor

```
[KOTU YOL]
Uretilen SQL:
SELECT id, ad, rol FROM kullanıcı WHERE ad = 'ayse' AND parola = '' OR '1'='1'
-> 3 satir dondu. GIRIS BASARILI:
    id=1 ad=ayse rol=kullanici
    id=2 ad=mehmet rol=kullanici
    id=3 ad=admin rol=yonetici
^ Parola bilinmeden giris yapildi: sorgu yapisi degisti.
[IYI YOL]
-> Sonuc yok. GIRIS REDDEDILDI.
^ Girdi bir DEGER olarak arandi; oyle bir parola yok: RED.
```

Bölüm 3'teki adımları burada takip edin: `parola = ''` yazıldıktan sonra saldırganın ilk `'` karakteri dize sabitini kapatır; ardından gelen `OR '1'='1'` metni SQL operatörü olarak okunur. `'1'='1'` her zaman **doğru** olduğu için `AND` bağlacının sağ tarafı her satır için doğrudur; sorgu fiilen `WHERE ad='ayse' OR doğru` olur ve **bütün tablo** döner. İyi yolda ise saldırganın yazdığı bütün dize (tırnak, `OR`, eşitlikler dahil) veritabanına tek bir metin **değeri** olarak gider; böyle bir parolaya sahip kimse olmadığı için sonuç boştur.

ADIM 3 — SALDIRI: ad alanından yönetici satirini cekme, ad = ' OR rol='yonetici' --

```
[KOTU YOL]
Uretilen SQL:
SELECT id, ad, rol FROM kullanıcı WHERE ad = '' OR rol='yonetici' --' AND parola = 'farketmez'
-> 1 satir dondu. GIRIS BASARILI:
    id=3 ad=admin rol=yonetici
^ Baska kullanicinin (yonetici) satiri sizdirildi.
[IYI YOL]
-> Sonuc yok. GIRIS REDDEDILDI.
^ Boyle bir ad yok: sizinti yok.
```

Burada saldırgan hem sorgu yapısını `OR` ile değiştirdi **hem de** `--` ile sorgunun geri kalanını (parola denetimini) SQL yorumuna çevirip devre dışı bıraktı; parola alanına ne yazıldığının artık **hiçbir önemi yok**. Parametrelili sorguda bu dize (`'`, `OR`, `--` dahil) tek bir metin değeri olarak `ad` sütunuyla karşılaştırılır; böyle bir kullanıcı adı olmadığı için sonuç boştur.

Demo 1 — SQL enjeksiyonu: dize birleştirme ve parametrelili sorgu

Demo 1 · [code/week-05/01-sql-enjeksiyonu](#) · CWE-89 · IDS00-J · Tarif 3.11

Demo, demo klasöründe sentetik kullanıcılarla küçük bir SQLite veritabanı oluşturur ve aynı giriş/arama sorgusunu iki yolla kurar: dize birleştirmeye ve parametrelili sorguyla. Ana bölüm Python'un yerleşik `sqlite3` modülüyle çalışır ve hiçbir şey indirmez; isteğe bağlı Java bölümü aynı fikri gerçek JDBC `PreparedStatement` ile gösterir (sürücü `hazirla` betiğiyle demo klasörüne indirilir).

```
cd code\week-05\01-sql-enjeksiyonu
.\demo.ps1
```

```
cd code/week-05/01-sql-enjeksiyonu
sh demo.sh
```

Adım	Ne olur?	Ders
1	Dürüst giriş: iki yol da çalışır	Hatalı kod normal durumda doğru görünür
2	Parola alanına ' OR '1'='1	Dize birleştirmede parola bilinmeden giriş yapılır
3	Arama alanına sorgunun yapısını değiştiren girdi	Başka kullanıcının (yöneticinin) satırı sızar
4	Aynı girdiler parametrelili sorguyla	Girdi yalnız değerdir; saldırı reddedilir

? Değerlendirici nasıl test eder?

Kaynak kodda sorgu metnine değişken birleştiren her yeri arar ("SELECT + +, String.format ile SQL, createStatement). Dinamik testte her girdi alanına tek tırnak, yorum işareti ve mantıksal ifadeler koyar; hata iletilerinde, yanıt sürelerinde ve dönen kayıt sayısında değişiklik arar. Otomatik araçlar (ör. sqlmap) yalnız **izin verilen** test ortamlarında kullanılır.

5. Komut enjeksiyonu

Uygulamalar bazen bir işi dış bir programa yaptırır: bir resmi dönüştürmek, bir arşivi açmak, bir ağ adresini sınamak. Komut tek bir **dize** olarak bir **kabuğa** verilirse, dizedeki kabuk karakterleri (; , && , | , ` , \$()) yeni komutlar başlatır:

Komut çalıştırma: kabuk var mı yok mu?

komut enjeksiyonunun tek kaynağı: araya giren kabuk

Kabuklu (tehlikeli)

```
Runtime.exec("sh -c ...")
```

kabuk ; | && > yorumlar
girdi KOMUT olur

ProcessBuilder (güvenli)

```
new ProcessBuilder(
    "prog", arg1, arg2)
```

argümanlar AYRI geçer
kabuk hiç çalışmaz

Yine de argüman enjeksiyonuna dikkat: -- ile seçenek/değer sınırını işaretleyin.

Hatalı: kabuk üzerinden dize

```
String kullanıcı = istek.getParameter("ad");
Runtime.getRuntime().exec(new String[]{"sh", "-c", "echo Merhaba " + kullanıcı});
/* ad = "ayse; <başka bir komut>" → kabuk iki komut çalıştırır */
```

Windows'ta aynı durum `cmd /c` ile ve `&` karakteriyle oluşur. Java'nın tek dize alan `Runtime.exec(String)` biçimi ayrıca dizeyi boşluklardan kendi kurallarıyla böler; tırnaklarla oynayarak argümanları kaydırmak mümkündür (IDS07-J).

Kabuk girdiyi adım adım nasıl ayrıştırır?

Bölüm 3'teki genel ilkeyi kabuğa (`sh`, `bash`, `cmd`) uygulayalım. `kullanici` değişkeninin içeriği `"ayse; echo SIZDI"` olsun; Java bu dizeyi birleştirip kabuğa şu **tek metni** verir: `echo Merhaba ayse; echo SIZDI`. Kabuk bunu şöyle işler:

1. Kabuk, aldığı metni **komut ayırıcı** karakterlere göre (`;`, `&&`, `||`, satır sonu) önce **komutlara böler**. Bu, tıpkı SQL ayrıştırıcısının `'` karakterinde durum değiştirmesi gibi, kabuğun kendi söz dizimi kuralıdır.
2. `;` karakterine kadar olan kısım (`echo Merhaba ayse`) **birinci komut** olarak ayrılır.
3. `;` 'den sonraki kısım (`echo SIZDI`) **ikinci, bağımsız bir komut** olarak ayrılır.
4. Kabuk, iki komutu da **sırayla çalıştırır** — sanki kullanıcı terminale iki ayrı satır yazmış gibi.
5. Uygulamanın kendisi tek bir `exec` çağırısı yaptığını sanır; ama kabuk açısından ortada **iki** komut vardır, çünkü kabuk metni kendi kurallarıyla yeniden ayrıştırmıştır.

Kritik nokta yine aynıdır: kabuk **hata yapmıyor**, `;` karakterini tam olarak söz dizimi kuralına göre okuyor. Java'nın "bu bir kullanıcı adıydı" bilgisi, dize kabuğa ulaştığı anda kaybolmuştur.

Doğrusu: kabuksuz, argüman listesiyle

Doğru: ProcessBuilder + izin listesi

```
private static final Pattern AD = Pattern.compile("^[A-Za-z0-9_]{1,32}$");

if (!AD.matcher(kullanici).matches()) {
    throw new IllegalArgumentException("gecersiz ad"); // varsayılan: reddet
}
Process p = new ProcessBuilder("/usr/bin/printf", "Merhaba %s\n", kullanici)
    .redirectErrorStream(true)
    .start();
```

1. **Kabuk yok:** `ProcessBuilder` programı doğrudan başlatır; `;` ya da `&` yalnız argümanın içindeki karakterlerdir.
2. **Tam yol:** 1. haftadaki PATH saldırısına karşı programın tam yolu verilir.
3. **İzin listesi:** Argüman, çalıştırılmadan önce beklenen biçime göre doğrulanır.
4. **En iyisi:** Dış programa hiç gerek bırakmamak. Bir resim dönüştürmek için bir kütüphane, bir adresi sınamak için `InetAddress.isReachable` kullanmak, bütün sorunu ortadan kaldırır.



Argüman listesi her şeyi çözmez

Kabuk olmasa bile, çağrılan program kendi argümanlarını yorumlar. `-` ile başlayan bir kullanıcı girdisi, programa **seçenek** olarak geçebilir (argüman enjeksiyonu): ör. bir dosya adı yerine `--output=/baska/yer`. Çoğu program `--` ile "bundan sonrası seçenek değil" denmesini destekler; izin listesi de `-` ile başlayan girdileri reddetmelidir.



Sık yapılan hata: yalnız tehlikeli görünen karakterleri süzmek

Bazı geliştiriciler girdiden yalnız `;` ve `&` 'yi kaldırıp "temizledim" sanır. Kabuğun komut ayırıcıları bu ikisiyle sınırlı değildir: `|`, ```, `$( )`, satır sonu karakteri (`\n`), hatta `>` ve `<` (dosyaya yönlendirme) de kabuk için özeldir. Kara liste yaklaşımı (belirli karakterleri yasakla) her zaman bir sonraki unutulmuş karaktere açıktır.

✓ Kural

Komut enjeksiyonunun birincil savunması **kabuğu hiç çağırmamaktır** (`ProcessBuilder`, `subprocess` argüman listesi). Kara liste değil, izin listesi (beklenen karakter kümesiyle eşleşme) ikincil bir katmandır.

İşlenmiş örnek: Demo 2'nin çıktısını satır satır okumak

Demo 2'nin Python bölümü (`komut.py`) bir "selamlama aracı"nı önce kabuk üzerinden (`shell=True`), sonra argüman listesiyle (`shell=False`) çalıştırır. WSL/Linux'ta gerçek çıktı şöyledir:

ADIM 1 — Durust girdi: `ad = 'Ayse'`

```
[KOTU YOL]
Kabuga giden komut:
echo "ARAC CIKTISI: Merhaba" Ayse
| ARAC CIKTISI: Merhaba Ayse
[IYI YOL]
Arguman listesi:
[python, -c, ...] ... 'Ayse'
| ARAC CIKTISI: Merhaba Ayse
```

ADIM 2 — SALDIRI: `ad = 'Ayse; echo SIZDI-KOMUT-ENJEKSİYONU'`

```
[KOTU YOL]
Kabuga giden komut:
echo "ARAC CIKTISI: Merhaba" Ayse; echo SIZDI-KOMUT-ENJEKSİYONU
| ARAC CIKTISI: Merhaba Ayse
| SIZDI-KOMUT-ENJEKSİYONU
^ 'SIZDI...' satiri = enjekte edilen komut kostu.
[IYI YOL]
Izin listesi REDDETTI ([A-Za-z0-9_]+$):
girdi = Ayse; echo SIZDI-KOMUT-ENJEKSİYONU
-> Program hic calistirilmadi.
^ Izin listesi bosluk/;/& gordu ve reddetti.
```

Kötü yolda kabuk, yukarıdaki beş adımı izleyerek dizeyi ; üzerinden ikiye böldü ve **iki ayrı komut** çalıştırdı; ikinci satır (`SIZDI-...`) enjekte edilen komutun **gerçekten çalıştığının** kanıtıdır (demo zararsız bir `echo` kullanır). İyi yolda `ProcessBuilder` / `subprocess` hiçbir kabuk çağırmaz; girdi tek bir argümandır ve izin listesi (`[A-Za-z0-9_]+`) boşluk ile ; içeren bu girdiyi çalıştırmadan **önce** reddeder.

Demo 2 — Komut enjeksiyonu

i Demo 2 · `code/week-05/02-komut-enjeksiyonu` · CWE-78 · IDS07-J · Tarif 1.7–1.8

Bir "selamlama aracı" kullanıcı adını dış bir programa iletir. Hatalı sürüm komutu tek bir dize olarak kabuğa verir (`sh -c, cmd /c, Java'da Runtime.exec(String)`); girdideki ; ya da & ikinci bir komut başlatır. Enjekte edilen komut **yalnız zararsız bir echo** 'dur. Doğru sürüm `ProcessBuilder` (Java) ve `subprocess` argüman listesi (Python) kullanır ve girdiyi bir izin listesiyle doğrular. Aynı ders iki dilde gösterilir.

```
cd code\week-05\02-komut-enjeksiyonu
.\demo.ps1
```

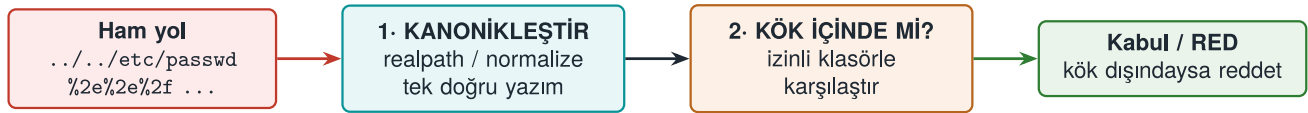
```
cd code/week-05/02-komut-enjeksiyonu
sh demo.sh
```

6. Yol geçişi (Tarif 3.7)

Bir dosya sunucusu yalnız belirli bir klasörün altındaki dosyaları vermelidir. İstenen dosya adı doğrudan bu klasörün yoluna eklenirse, `../` dizileri kökün **dışına** çıkar:

Yol geçişini kapatan iki adım

sıra önemli: önce kanonikleştir, SONRA denetle



Ham dize üzerinde yapılan denetim kodlanmış biçimlerle atlatılır.

Hatalı

```
Path kok = Paths.get("/srv/veri");
Path dosya = kok.resolve(istek); // istek = "../etc/passwd"
return Files.readAllBytes(dosya); // kökün dışındaki dosya okunur
```

Yol normalleştirme adım adım: `../` neden köke geri çıkarır?

`resolve() / normalize()` bir yolu tek bir **yığın** (stack) algoritmasıyla işler. `kok = "/srv/veri"` ve `istek = "../etc/passwd"` olsun. Dosya sistemi bu isteği aşağıdaki gibi, segment segment işler:

Adım	Okunan segment	Yığının durumu	Açıklama
0	(başlangıç)	[srv, veri]	Kök yoldan başlanır
1	..	[srv]	Bir üst segment (veri) yığından çıkarılır (pop)
2	..	[]	Bir üst segment daha (srv) yığından çıkarılır
3	etc	[etc]	Normal segment yığına eklenir (push)
4	passwd	[etc, passwd]	Normal segment eklenir

Sonuç: `/etc/passwd` — kökün (`/srv/veri`) tamamen dışında bir yol. `..` karakterleri, ayrıştırıcı için "bir üst klasöre çık" komutudur; SQL'deki `'` nasıl "dize sabitini kapat" komutuysa, dosya sisteminde de `..` aynı rolü oynar. Bu yüzden ham dizede yalnız `".."` arayan bir denetim (`istek.contains("..")`) yeterli değildir: `..%2f` (URL kodlanmış), `....//` (iç içe

yazım) gibi biçimler bu aramayı atlatırken, **aynı yığın algoritmasıyla** işlendiğinde yine kökün dışına çıkabilir. Doğru sıra bu yüzden önce **normalleştir** (yığın algoritmasını tamamen çalıştır), sonra **sonucu** denetlemektir — ara adımları değil, nihai yığın içeriğini kontrol ederiz.

Doğrusu: önce kanonikleştir, sonra kökün içinde mi diye bak

Doğru: kanonik yol + kök denetimi (FIO16-J)

```
Path kok = Paths.get("/srv/veri").toRealPath();
Path aday = kok.resolve(istek).normalize(); // istek mutlak yolsa resolve onu olduğu gibi döndürür
if (!aday.startsWith(kok)) throw new SecurityException("kok dışı");
Path gercek = aday.toRealPath(); // sembolik bağlantıları da çözer
if (!gercek.startsWith(kok)) throw new SecurityException("kok dışı");
return Files.readAllBytes(gercek);
```

Sıra önemlidir: önce **kanonik** (tek, kesin) yol elde edilir, sonra denetlenir. Denetimi ham dizeye yapmak (`istek.contains("..")`) yetmez; `..%2f`, `....//`, Windows'ta `..\` ve sürücü harfli mutlak yollar, Unicode biçimleri ve **sembolik bağlantılar** denetimi atlatabilir. `toRealPath()` sembolik bağlantıları da gerçek hedeflerine çözer; ikinci haftadaki TOCTOU (denetim ile kullanım arasındaki zaman) sorunu hâlâ geçerli olduğu için dosya açıldıktan sonra da kök içinde olduğunun doğrulanması en sağlamıdır.



Arşivlerdeki yol geçişi: Zip Slip

Bir ZIP ya da TAR arşivini açarken, arşivdeki dosya adları da güvenilmez girdidir. `../../../../x` adlı bir girdi, arşivi açan programın hedef klasörünün dışına yazılabilir. Arşiv açan her kod, her girdinin hedef yolunu yukarıdaki gibi kanonikleştirip kök içinde olduğunu denetlemelidir.



Sık yapılan hata: yalnız `..` dizesini aramak

`if (istek.contains(".."))` satırı öğrencilerin en sık yazdığı "düzeltmedir". Ama `..%2f` (URL kodlanmış nokta ve eğik çizgi, sunucuya ulaşmadan önce kod çözülmüşse `../` 'ye döner), Windows'ta `..\`, sürücü harfli mutlak yollar (`C:\...`) ve kök klasörün **içindeki** bir sembolik bağlantının kök dışını göstermesi, bu tek satırlık denetimin hiçbirini yakalamaz. `contains("..")` ayrıca meşru bir dosya adında (`rapor..v2.txt`) yanlış pozitif üretebilir.



Kural

Yol geçişine karşı tek geçerli desen: **önce normalleştir (kanonikleştir), sonra sonucun kök içinde kaldığını denetle, gerekiyorsa dosya açıldıktan sonra tekrar denetle**. Ham dizede desen aramak hiçbir zaman yeterli değildir.

İşlenmiş örnek: Demo 3'ün çıktısını satır satır okumak

Demo 3'ün Python bölümü (`yol.py`) `cikti/veri/` klasörünü **kök** kabul eder; kök dışında (`cikti/gizli.txt`) sentetik bir "yönetici notu" bulunur. Gerçek çıktı:

ADIM 1 — Mesru istek: 'rapor.txt'

```
[KOTU YOL]
  Cozulen yol: ../cikti/veri/rapor.txt
  OKUNDU -> Herkese acik rapor (sentetik).
[IYI YOL]
  Cozulen yol: ../cikti/veri/rapor.txt
  OKUNDU -> Herkese acik rapor (sentetik).
```

ADIM 2 — SALDIRI: '../gizli.txt' (kok disi)

```
[KOTU YOL]
  Cozulen yol: ../cikti/gizli.txt
  OKUNDU -> GIZLI: sentetik yonetici notu (kok disinda).
  ^ Kok disindaki gizli dosya sizdirildi.
[IYI YOL]
  REDDEDILDI: kok disina cikiyor -> ../cikti/gizli.txt
  ^ resolve() + kok denetimi engelledi.
```

Kötü yolda `KOK / "../gizli.txt"` ifadesi, yukarıdaki yığın algoritmasıyla `veri` segmentini pop'lar ve doğrudan `cikti/gizli.txt` 'ye ulaşır — kökün **bir üst klasörüne** çıkmıştır. İyi yolda aynı normalleştirme yapılır, ama sonuç kök (`cikti/veri`) ile başlamadığı için istek çalıştırılmadan **reddedilir**.

Demo 3 — Yol geçişi

Demo 3 · `code/week-05/03-yol-gecisi` · CWE-22 · FIO16-J · Tarif 3.7

Bir "dosya sunucusu" yalnız `cikti/veri/` altındaki dosyaları vermelidir. Hatalı sürüm isteği doğrudan köke ekler ve `../gizli.txt` isteği kök dışındaki **sentetik** gizli dosyayı sızdırır. Doğru sürüm yolu kanonikleştir (`normalize()` + `toRealPath()`), Python'da `resolve()` ve sonucun kök içinde kaldığını doğrular; mutlak yol ve sürücü harfi de reddedilir. Windows'ta `..\` ve `../` ayırmalarının ikisi de işlenir.

```
cd code\week-05\03-yol-gecisi
.\demo.ps1

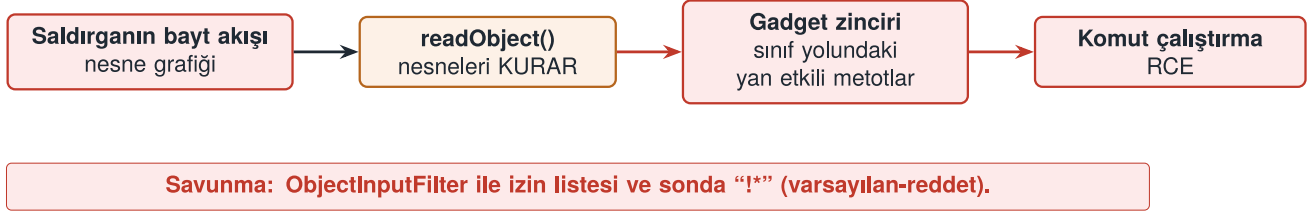
cd code/week-05/03-yol-gecisi
sh demo.sh
```

7. Güvensiz seri durumdan çıkarma

Serileştirme (serialization), bir nesneyi bayt dizisine çevirip saklamak ya da ağdan göndermektir; **seri durumdan çıkarma** (deserialization) ise tersidir. Java'nın yerleşik `ObjectInputStream` mekanizması, bayt akışında adı yazan **herhangi bir** serileştirilebilir sınıfın nesnesini oluşturur ve bu sırada o sınıfın bazı metotlarını (`readObject`, `readResolve` gibi) çalıştırır.

Güvensiz seri durumdan çıkarma

saldırıcı kod yazmaz — VAR OLAN sınıfları zincirler



Sorun şudur: bayt akışı güvenilir bir kaynaktan geliyorsa, **hangi sınıfların oluşturulacağını saldırıcı seçer**. Uygulamanın sınıf yolunda (classpath) bulunan kütüphanelerdeki masum görünen sınıflar, oluşturulurken birbirini tetikleyecek biçimde zincirlenebilir. Gerçek olaylarda bu zincirler uzaktan kod çalıştırmaya kadar gitmiştir; 2015'te yaygın bir ortak kütüphanedeki sınıflar üzerinden birçok kurumsal sunucu ürününü etkileyen bir dizi zafiyet bu yüzden ortaya çıktı. Bu nedenle CWE-502 ("güvenilmeyen verinin seri durumdan çıkarılması"), OWASP Top 10'daki "yazılım ve veri bütünlüğü hataları" kategorisinin parçasıdır.

Bu derste zincir yazmıyoruz

Saldırı zincirlerinin nasıl kurulduğu bu dersin konusu değildir. Bilmemiz gereken tek şey: **güvenilmeyen bir bayt akışını filtresiz seri durumdan çıkarmak, saldırıcıya sınıf seçme yetkisi vermektir**. Savunma bu tek cümleye dayanır.

Bayt akışı nasıl bir sınıf adı taşır? Mekanizma adım adım

Saldırı zincirinin **kendisini** yazmıyoruz ama saldırıcının elindeki yetkinin **nereden** geldiğini anlamak savunmayı da anlamlı kılar. `ObjectOutputStream`'in ürettiği bayt akışının başı sabit bir biçime sahiptir:

1. İlk iki bayt her zaman `AC ED` — "stream magic" (bu akışın bir Java nesne akışı olduğunu belirten imza).
2. Sonraki iki bayt `00 05` — akış biçiminin sürümü.
3. Ardından bir **nesne bloğu** gelir; bu blok önce bir **sınıf tanımlayıcısı** (class descriptor) içerir: sınıfın **tam adı** (`com.ornek.Ayar` gibi), uzunluk öneki ile birlikte **düz metin** olarak akışta durur.
4. `ObjectInputStream.readObject()` çalıştığında, önce bu ismi okur, sonra `Class.forName(isim)` benzeri bir çağrıyla sınıfı sınıf yolunda (classpath) **arayıp yükler** — henüz hiçbir tür denetimi yapılmamıştır, çünkü beklenen türün ne olduğunu akışın kendisi söylemektedir.
5. Sınıf yüklendikten sonra JVM bu sınıftan bir nesne **ayırır** (adi bir kurucu çağırısı olmadan, seri durumdan çıkarmaya özgü bir mekanizmayla) ve akıştaki alan değerlerini bu nesneye yazar; sınıfın tanımladığı `readObject` / `readResolve` gibi özel metotlar varsa bunlar da bu sırada **çalıştırılır**.

Adım 3–4'ün sonucu şudur: **hangi sınıfın yükleneceğine akış gönderen karar verir**, uygulama değil. Akış güvenilir bir kaynaktan geliyorsa, sınıf yolundaki *herhangi bir* sınıf adı yazılabilir; o sınıfın `readObject` / `readResolve` metotları çalışırken yaptığı her şey (dosya açmak, başka bir nesne oluşturmak, bir metot çağırmak) saldırıcının kontrolündedir. Bir dizi böyle sınıfın birbirini **zincirleme** tetiklemesi (gerçek olaylardaki "gadget zinciri"), sonunda keyfi kod çalıştırmaya kadar gidebilir — zincirin nasıl kurulduğunu **yazmıyoruz**, ama "neden mümkün olduğunu" Adım 3–4 tam olarak açıklar.

**Sık yapılan hata: 'kendi sınıflarım seri durumdan çıkıyor, güvenliyim' sanmak**

Filtre olmadan `ObjectInputStream`, akışta adı yazan **her** `Serializable` sınıfı kabul eder — yalnız sizin yazdığınız sınıfları değil, sınıf yolundaki **bütün** bağımlılıklarınızın (ve onların bağımlılıklarının) sınıflarını da. Uygulamanız hiç tehlikeli bir `readObject` yazmasa bile, kullandığınız bir kütüphanede böyle bir sınıf varsa, akışa onun tam adını yazmak yeterlidir.

**Kural**

Güvenilmeyen bir bayt akışını `ObjectInputStream.readObject()` ile açmadan önce mutlaka bir `ObjectInputFilter` takın ve deseni **her zaman** `!*` ile bitirin. Mümkünse Java serileştirmesini hiç kullanmayın; JSON/Protobuf gibi biçimler sınıf adını akışa hiç yazmaz, yalnız veri alanlarını taşır.

Savunma katmanları

Sıra	Önlem	Açıklama
1	Hiç kullanmamak	Güvenilmeyen veri için Java serileştirmesi yerine veri biçimleri (JSON, Protobuf) kullanın; bunlar yalnız veri taşır, sınıf seçtirmez
2	İzin listesi filtresi	Java 9'dan beri <code>ObjectInputFilter</code> (JEP 290), Java 17'den beri bağlama göre filtre fabrikası (JEP 415): yalnız beklenen sınıflar
3	Sınırlar	Akışın derinliği, dizi boyutu, toplam bayt sayısı ve nesne sayısı sınırlanır (hizmet engellemeye karşı)
4	Bütünlük	Serileştirilmiş veri saklanıyor ya da taşınıyorsa HMAC ile imzalanır; imza doğrulanmadan çözülmez
5	Bağımlılık hijyeni	Sınıf yolunda kullanılmayan kütüphane bırakmamak (14. bölüm)

İzin listesi filtresi (SER12-J)

```
ObjectInputFilter filtre = ObjectInputFilter.Config.createFilter(
    "com.ornek.Ayar;java.base/*;maxdepth=5;maxarray=1000;maxbytes=65536;!*");
try (ObjectInputStream in = new ObjectInputStream(akis)) {
    in.setObjectInputFilter(filtre);
    Ayar a = (Ayar) in.readObject();           // beklenmeyen sınıf → InvalidClassException
}
```

Desen soldan sağa okunur: `com.ornek.Ayar` ve `java.base` modülündeki sınıflara izin ver, sınırları uygula, **geri kalan her şeyi reddet** (`!*`). Son öğe varsayılan-reddet ilkesidir; unutulursa filtre bir kara listeye dönüşür.

İşlenmiş örnek: Demo 7'nin çıktısını satır satır okumak

Demo 7'nin Java kodu (`SeriDemo.java`), iki zararsız sınıf tanımlar: **beklenen** `Ayar` (ad + değer taşır) ve **beklenmeyen** `BaskaSınıf` (gerçek bir saldırıda bir "gadget" olurdu; burada yalnızca bir dize taşır). Her ikisi de serileştirilip, önce filtresiz,

sonra filtreli çözülür. Gerçek çıktı:

ADIM 1 — FİLTRESİZ cozme: her sınıf kabul edilir

```
[beklenen Ayar] KABUL -> Ayar(ad=zaman-asimi, deger=30) (Ayar)
[beklenmeyen BaskaSinif] KABUL -> BaskaSinif(yuk=beklenmeyen-sinif) (BaskaSinif)
^ Filtre olmadan gelen HER sınıf olusturulur;
  gercekte bu bir gadget zinciri olabilirdi.
```

ADIM 2 — ObjectInputFilter ile: allow-list

```
[beklenen Ayar] KABUL -> Ayar(ad=zaman-asimi, deger=30)
[beklenmeyen BaskaSinif] REDDEDILDI (filtre): beklenmeyen sınıf engellendi.
```

Filtre "SeriDemo\$Ayar;java.base/*;!*" yalnız `Ayar` sınıfına ve Java'nın kendi çekirdek sınıflarına izin verir. Adım 1'deki mekanizmayı hatırlayın: filtre, sınıf **yüklenmeden önce** akıştan okunan adı denetler; `BaskaSinif` listede olmadığı için `readObject` çağrısı `InvalidClassException` ile durur — sınıf hiç örneklenmez, hiçbir metodu çalışmaz.

Demo 7 — Güvenli seri durumdan çıkarma

Demo 7 · code/week-05/07-deserializasyon · CWE-502 · SER12-J

Demo **hiçbir saldırı zinciri içermez**. Filtresiz sürüm akıştaki her sınıfı oluşturur: beklenen `Ayar` da, beklenmeyen `BaskaSinif` da kabul edilir. Filtreli sürüm `ObjectInputFilter` ile yalnız beklenen sınıflara izin verir; beklenmeyen sınıf `InvalidClassException` ile reddedilir.

```
cd code\week-05\07-deserializasyon
.\demo.ps1
```

```
cd code/week-05/07-deserializasyon
sh demo.sh
```

Değerlendirici nasıl test eder?

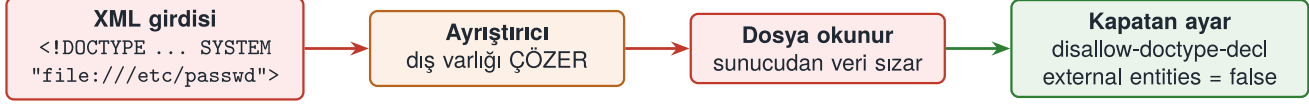
Kaynak kodda `ObjectInputStream`, `readObject`, XML tabanlı nesne çözücüler ve "polimorfik tür" özelliği açık JSON kütüphaneleri arar; her kullanımda verinin kaynağını sorar. Filtre varsa deseninin `!*` ile bittiğini denetler. Ağ trafiğinde ya da dosyalarda Java serileştirmesinin sihirli baytlarını (`AC ED 00 05`) arar.

8. XML, şablon ve diğer enjeksiyonlar

Aynı "veri ile komutun karışması" hatası her yorumlayıcıda karşımıza çıkar. En sık görülen dördü:

XXE: XML dış varlık saldırısı

ayrıştırıcının “yardımsever” varsayılını



Kural: XML/YAML/şablon ayrıştırıcılarında gereksiz yetenekleri KAPATIN.

Tür	Nasıl oluşur?	Savunma	CWE
XML dış varlık (XXE)	XML ayrıştırıcısı, belgedeki <code><!DOCTYPE></code> tanımlarıyla yerel dosyaları ya da ağ adreslerini okur	Ayrıştırıcıda DTD ve dış varlıkları kapatmak	611
Siteler arası betik (XSS)	Kullanıcı verisi HTML sayfasına kaçışsız yazılır; başka kullanıcının tarayıcısında betik çalışır	Şablon motorunun bağlama göre otomatik kaçışı, İçerik Güvenliği Politikası (CSP)	79
Şablon enjeksiyonu	Kullanıcı verisi şablonun kendisi olarak işlenir	Kullanıcı verisini yalnız şablon değişkeni olarak vermek	1336
LDAP / XPath enjeksiyonu	Dizin ya da XML sorgusu dize birleştirmeye kurulum	Parametrelili arayüzler, özel karakterlerin kaçışı	90, 643

XXE mekanizması adım adım (yük yazmadan)

Bu derste çalışır bir XXE yükü yazmıyoruz

Deserialization bölümünde olduğu gibi, burada da gerçek bir saldırı belgesi vermiyoruz. Bilmemiz gereken, XML ayrıştırıcısının **hangi adımı** kötüye kullanılabilir hale getirdiğidir; savunma (aşağıdaki kod) bu adımı kapatmaktan ibarettir.

XML standardı, bir belgenin başında bir **DTD** (Document Type Definition) tanımlamasına ve bu DTD içinde özel kısaltmalar (**varlık**, entity) tanımlanmasına izin verir — tıpkı bir metin editöründeki “ara-değiştir” kısayolu gibi: belge içinde `&kisaltma;` yazılan her yere, DTD’de tanımlanan **gerçek içerik** yerleştirilir. Sorun, bu “gerçek içeriğin” **dış bir kaynaktan** (`SYSTEM` anahtar sözcüğüyle işaretli bir dosya yolu ya da ağ adresi) gelebilmesidir:

1. Ayrıştırıcı belgeyi okurken önce **DTD bloğunu** işler; DTD içindeki her `<!ENTITY ad SYSTEM "kaynak">` tanımını bir **kısaltma tablosuna** kaydeder (`ad` → `kaynak`).
2. Ayrıştırıcı belgenin gövdesini işlerken `&ad;` biçiminde bir referansla karşılaşır, kısaltma tablosundaki `kaynak` 'ı **açar** (dosyayı okur ya da ağ isteği yapar) ve içeriğini `&ad;` 'nin yerine **yerleştirir**.

3. Bu yerleştirme, uygulamanın kendi mantığı devreye girmeden, **ayırıştırma sırasında** olur; uygulama XML'i "okumayı bitirdiğinde" varlık zaten genişletilmiş, dosya içeriği belge metnine karışmıştır.
4. Eğer uygulama bu genişletilmiş metni herhangi bir biçimde kullanıcıya geri gösteriyorsa (bir hata iletilsinde, bir yanıt alanında), dosyanın içeriği **sızdırılmış** olur; kaynak bir ağ adresiyse bu aynı zamanda sunucunun iç ağa istek göndermesi (SSRF, [3. hafta](#)) anlamına gelir.

Kritik nokta yine aynı: ayırıştırıcı hata yapmıyor, DTD standardını **tam olarak** uyguluyor. Aşağıdaki sağlamlaştırma (hardening), bu dört adımın **birinci adımını** (DTD'nin hiç işlenmesini) ve **ikinci adımını** (dış kaynaklı varlıkların açılmasını) baştan kapatır; böylece 3–4. adımlara hiç sıra gelmez.



Sık yapılan hata: yalnız kullanıcı girdisini doğrulayıp XML ayırıştırıcısını olduğu gibi bırakmak

Girdi doğrulama (beyaz liste, uzunluk sınırı) XML'in **içeriğine** bakar; ama XXE, belgenin **yapısal** bir özelliğini (DTD/varlık tanımı) kötüye kullanır. "İçerik güvenli görünüyordu" demek yeterli değildir — varlık genişletme, ayırıştırıcı düzeyinde, uygulamanın hiçbir girdi denetimine uğramadan gerçekleşir.



Kural

Uygulamanız DTD ya da dış varlık kullanmıyorsa (çoğu web servisi kullanmaz), ayırıştırıcıda bunları **her zaman** kapatın (`disallow-doctype-decl`, dış varlıkları reddetme). Bu, "ihtiyacınız olmayanı kapatın" ilkesinin (Bölüm 8'in girişindeki kural) en doğrudan uygulamasıdır.

XXE'ye karşı ayırıştırıcıyı sağlamlaştırmak (IDS17-J)

```
DocumentBuilderFactory f = DocumentBuilderFactory.newInstance();
f.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true); // DTD yok
f.setFeature("http://xml.org/sax/features/external-general-entities", false);
f.setFeature("http://xml.org/sax/features/external-parameter-entities", false);
f.setExpandEntityReferences(false);
f.setXIncludeAware(false);
```

Kural her seferinde aynıdır: **yorumlayıcının hangi özelliklerinin açık olduğunu bil, ihtiyacın olmayanı kapat, veriyi komut kanalından geçirme**. Kitabın Tarif 3.10'daki XSS önerileri (çıkıyı bağlama göre kodlamak, girdiyi beyaz listeye doğrulamak) bugünkü web çerçevelerinin varsayılan davranışı haline gelmiştir; ama bir şablonda "ham HTML" seçeneğini açmak ya da bir sayfayı dize birleştirerek üretmek aynı hatayı geri getirir.

XSS'te aynı mekanizma: tarayıcı HTML'i nasıl ayırıştırır?

XXE'deki "ayırıştırıcı, dış kaynaklı içeriği genişletir" fikrinin tam karşılığı, XSS'te **tarayıcının** HTML ayırıştırıcısıdır. "`<p>`" + `yorum` + "`</p>`" gibi bir dize birleştirmede, `yorum` değişkeni bir HTML **etiket açma karakteri** (`<`) içeriyorsa, tarayıcı bu karakteri gördüğü anda "metin durumundan" "etiket durumuna" geçer — SQL ayırıştırıcısının `'` karakterinde durum değiştirmesiyle birebir aynı mekanizma. Sonuç: kullanıcının yazdığı metnin bir kısmı, sayfanın **yapısının** (yeni bir etiket, dolayısıyla çalıştırılabilir bir betik etiketi) bir parçası olur ve bu HTML'i görüntüleyen **her kullanıcının** tarayıcısında çalışır — SQL enjeksiyonundan farkı, kurbanın veritabanı değil, sayfayı açan **başka bir kullanıcı** olmasıdır. Şablon motorlarının "otomatik kaçışı" tam olarak Bölüm 4'teki parametrelili sorguyla aynı işi yapar: kullanıcı verisini HTML **söz dizimi** olarak değil, salt **metin içeriği** olarak kodlar (`<` yerine `<`; yazar), böylece ayırıştırıcı hiçbir zaman "etiket durumuna" geçmez.

9. Yorumlanan diller: Python ve JavaScript'te aynı hatalar

izlence bu haftayı "Java ve yorumlanan diller" diye adlandırır. Python, JavaScript (Node.js), Ruby, PHP gibi dillerde de bellek güvenliği bedavadır; ama bu diller **çalışma anında kod üretip çalıştırmayı** Java'dan çok daha kolay hale getirir. Aynı hata sınıfları burada farklı adlarla karşımıza çıkar:

Yorumlanan dillerde aynı hatalar

Python, JavaScript, Ruby, PHP — kök aynı: veri ile kodun karışması

eval / exec

dizeyi KOD olarak çalıştırır — girdiyle asla birleştirilmez

pickle / unserialize

veriyi nesneye çevirirken kod çağırır — güvenilirmez veride kullanılmaz

os.system / shell=True

kabuk ayrıştırır — argüman dizisi kullanın

Düzenli ifade (ReDoS)

geri izleme üstel büyür — desen ve girdi uzunluğu sınırlanır

Dil değişir, kural değişmez: girdiyi asla yorumlayıcıya kod olarak verme.

Hata	Java	Python	JavaScript / Node.js
SQL enjeksiyonu	Statement + birleştirme	cursor.execute(f"... {ad}")	Şablon dizesiyle sorgu
Doğrusu	PreparedStatement	cursor.execute("... ?", (ad,))	Sürücünün parametre bağlama arayüzü
Komut enjeksiyonu	Runtime.exec(String)	os.system, subprocess.run(..., shell=True)	child_process.exec
Doğrusu	ProcessBuilder	subprocess.run([...]) (liste, shell=False)	child_process.execFile / spawn (dizi)
Kod çalıştırma	Yansıma, betik motorları	eval, exec	eval, new Function, vm
Güvensiz seri durumdan çıkarma	ObjectInputStream	pickle.loads, yaml.load (güvensiz yükleyici)	Nesne çözen kütüphaneler; prototip kirlenmesi
Doğrusu	Filtre / JSON	json.loads, yaml.safe_load	JSON.parse + şema doğrulama

eval ve arkadaşları: veriyi kod olarak çalıştırmak

Yorumlanan dillerin en tehlikeli özelliği, bir dizeyi **kod olarak** çalıştırabilmeleridir. Kullanıcıdan gelen bir dizinin `eval` 'e ulaşması, enjeksiyonun en doğrudan biçimidir: yorumlayıcının kendisi komut kanalıdır.

Hatalı ve doğru: kullanıcının girdiği bir sayıyı okumak

```
deger = eval(girdi) # HATALI: girdi herhangi bir Python ifadesi olabilir

import ast
deger = ast.literal_eval(girdi) # Daha iyi: yalnız sayı, dizge, liste gibi SABİTLERİ kabul eder
deger = int(girdi) # En iyisi: beklenen türü doğrudan ayrıştır, hatayı yakala
```

Kural basittir: **kullanıcı verisi** `eval`, `exec`, `new Function` **ya da bir betik motoruna asla ulaşmamalı**. Bir hesap makinesi ya da kural motoru gerekiyorsa, yalnız izin verilen işlemleri tanıyan küçük bir ayrıştırıcı yazılır.



Sık yapılan hata: 'girdiyi süzdüm, sonra eval ettim' demek

Tehlikeli sözcükleri (`import`, `os`, `__`) aramak ve bulunmazsa `eval` etmek yaygın ama kırılğan bir savunmadır: Python'da aynı işlevi başka adlarla çağırmanın (dolaylı erişimlerle) yolları vardır ve bu liste hiçbir zaman tam olmaz. `eval` 'in kendisi kod kanalıdır; kanalın önüne süzgeç koymak, kanalı **kapatmaktan** daha zayıftır.



Kural

Kullanıcı girdisini bir dilin tam yorumlayıcısına hiç vermeyin. Yalnız sabitleri kabul eden bir ayrıştırıcı (`ast.literal_eval`) ya da beklenen türü doğrudan ayrıştıran bir fonksiyon (`int`, `float`) kullanın; bu ikisi kod **çalıştırmaz**, yalnız veri **okur**.

Seri durumdan çıkarma: `pickle` ve `YAML`

Python'un `pickle` modülü, Java serileştirmesiyle aynı sorunu taşır: çözülen veri, **hangi nesnelerin nasıl oluşturulacağını** belirler; belgelerinde de açıkça "güvenilmeyen veriyi asla pickle ile çözmeyin" yazar. `YAML` kütüphanelerinin tam yükleyicileri de benzer biçimde nesne oluşturabilir.

Güvenli seçimler

```
import json, yaml

veri = json.loads(metin) # yalnız veri: sözlük, liste, sayı, dizge
ayar = yaml.safe_load(metin) # yalnız temel YAML türleri
# pickle.loads(metin) # YALNIZ kendi ürettiğiniz ve HMAC ile doğruladığınız veride
```

Düzenli ifade ile hizmet engelleme (ReDoS)

Girdi doğrulamanın en yaygın aracı düzenli ifadelerdir; ama kötü yazılmış bir düzenli ifade kendisi bir saldırı yüzeyi olabilir. Geri izleme (backtracking) yapan motorlarda (Java, Python, JavaScript'in yerleşik motorları) iç içe tekrarlar içeren bir desen, özel olarak hazırlanmış bir girdide **üstel** sürede çalışır:

```
Desen: ^(a+)+$
Girdi: "aaaaaaaaaaaaaaaaaaaaaaaaaaaaa!" → motor, eşleşmeyi kanıtlamak için milyonlarca yol dener
```

Tek bir istek, sunucunun bir işlemcisini saniyelerce ya da dakikalarca meşgul edebilir (CWE-1333). Savunma:

1. İç içe tekrarlardan (`(a+)+`, `(a|a)*`, `(\w+\s?)*`) kaçının; deseni olabildiğince **kesin** yazın.
2. Girdinin uzunluğunu düzenli ifadede **önce** sınırlayın (4. haftadaki "önce uzunluk" ilkesi).
3. Mümkünse doğrusal zamanlı bir motor kullanın (RE2 ve türevleri).
4. Statik analiz araçlarının ReDoS kurallarını açın.

İşlenmiş örnek: geri izleme neden üstel büyür? Elle sayalım

Yukarıdaki "milyonlarca yol dener" cümlesini soyut bırakmayalım; küçük bir girdi üzerinde **gerçekten** kaç yol denendiğini sayalım. Desen `^(a+)+$` iki iç içe tekrardan oluşur: iç `(a+)` "bir ya da daha çok a" demektir, dış `(...)+` bu grubu "bir ya da daha çok kez" tekrarlar. Motor bir `a` dizisini, iç grubu **kaç parçaya böleceğine** dair bütün olasılıkları dener — buna bir sayının **birleşimleri** (compositions) denir.

Girdi olarak yalnız 4 tane `a` alalım (`"aaaa"`) ve motorun iç grubu bu 4 `a`'yı kaç farklı biçimde bölümleyebileceğini elle çıkaralım (her bölüm en az 1 `a` içermeli, çünkü `a+` boş olamaz):

#	Bölümleme (dış grubun kaç kez çalıştığı)	Parçalar
1	1 kez	<code>aaaa</code>
2	2 kez	<code>aaa + a</code>
3	2 kez	<code>aa + aa</code>
4	2 kez	<code>a + aaa</code>
5	3 kez	<code>aa + a + a</code>
6	3 kez	<code>a + aa + a</code>
7	3 kez	<code>a + a + aa</code>
8	4 kez	<code>a + a + a + a</code>

Tam **8** farklı bölümleme var. Genel kural: n tane `a`'nın kompozisyon sayısı 2^{n-1} 'dir ($n=4$ için $2^3=8$, tablo bunu doğruluyor). Girdi `"aaaa"` tek başına **eşleşir** (desen `$` ile biter, tam eşleşme başarılı olur), bu yüzden motor ilk denemede durabilir. Ama girdinin sonuna eşleşmeyi **bozan** tek bir karakter eklersek (`"aaaa!"`), `$` hiçbir zaman sağlanamaz; motor **8 bölümlemenin hepsini** dener, hepsi başarısız olur, sonunda "eşleşme yok" der — ama bu sonuca varmadan önce bütün yolları tüketmiştir.

n büyüdükçe deneme sayısı 2^{n-1} ile **üstel** büyür:

n (kaç a)	Deneme sayısı $2^{(n-1)}$	Yorum
4	8	Elle sayılabilir
10	512	Göz açıp kapayıncaya kadar
20	524.288	Hâlâ hızlı
30	≈ 536 milyon	Fark edilir gecikme
40	≈ 549 milyar	Saniyeler–dakikalar
50	≈ 562 trilyon	Pratikte "asla bitmez"

Saldırganın yapması gereken tek şey, n 'i yeterince büyütüp (ör. 40-50 karakter, hiç uzun değil) sonuna eşleşmeyi bozan **bir** karakter eklemektir; girdi kilobayt mertebesinde bile değildir, ama sunucunun tek bir işlem çekirdeğini dakikalarca kilitleyebilir. Bu, dördüncü haftadaki "küçük girdi, orantısız etki" fikrinin düzenli ifade karşılığıdır.



Sık yapılan hata: 'düzenli ifade zaten hızlıdır' varsayımı

Basit desenler (`^[0-9]+$` gibi) gerçekten doğrusal zamanda çalışır; sorun yalnız **iç içe tekrarlar**da (bir tekrarın içinde başka bir tekrar, ya da birbirinin alt kümesi olan iki alternatif) ortaya çıkar. Bir deseni test ederken yalnız "geçerli" girdilerle denemek bu hatayı **saklar**; ReDoS test edilmediği sürece üretimde ilk kez bir saldırganla karşılaşılır.



Kural

Bir düzenli ifadeyi kabul etmeden önce iki soru sorun: (1) aynı karakteri iki farklı grup **aynı anda** eşleştirebilir mi (belirsizlik = geri izleme riski)? (2) girdi uzunluğu düzenli ifadede **önce** sınırlı mı? İkisine de "hayır" ise desen ReDoS'a açıktır.

Prototip kirlenmesi (JavaScript)

JavaScript'te nesneler bir **prototip** zincirinden özellik devralır. Kullanıcıdan gelen bir JSON nesnesini derinlemesine birleştiren (merge) bir fonksiyon, `__proto__` gibi özel bir anahtara yazmaya izin verirse, bütün nesnelerin ortak prototipine yeni bir özellik eklenmiş olur; uygulamanın başka bir yerindeki `if (kullanici.yoneticisi)` denetimi beklenmedik biçimde doğru çıkabilir. Savunma: `__proto__`, `constructor` ve `prototype` anahtarlarını reddetmek, prototipsiz nesneler (`Object.create(null)`) ya da `Map` kullanmak, gelen JSON'u bir **şemayla** doğrulamak.



Dilden bağımsız kural

Hangi dili kullanırsanız kullanın, dört soru aynıdır: Veri bir yorumlayıcıya mı gidiyor (SQL, kabuk, `eval`, şablon)? Nesne oluşturan bir çözücüye mi gidiyor (`pickle`, Java serileştirmesi)? Uzunluğu ve biçimi sınırlı mı? Bağımlılığın kendisi güvenilir mi (14. bölüm)?

10. Bayt kodu ve tersine derleme

C/C++ derleyicisi kaynak kodu doğrudan makine koduna çevirir; bu sırada değişken adları, tipler ve yapının büyük kısmı kaybolur. Java derleyicisi ise kaynak kodu, Java'yı çalıştıran sanal makine olan **JVM**'nin (Java Virtual Machine) anladığı **bayt koduna** (`.class`) çevirir; Android'de bu bayt kodu ayrıca **DEX** biçimine dönüştürülür. Bayt kodu makine kodundan çok daha **yüksek düzeylidir**:

Bayt kod neden makine kodundan kolay okunur?

bu yüzden Java'da gizleme (ProGuard/R8) daha kritik

Java bayt kodu

sınıf/metot/alan ADLARI durur
tipler durur
yapı korunur
decompiler $\approx$ kaynağı verir

C/C++ makine kodu

adlar atılır
tipler kaybolur
optimizasyon yapıyı bozar
disassembler $\approx$ ham komut

Gizleme hatayı düzeltmez; yalnız “kolayca okunma” avantajını geri alır.

Bilgi	Makine kodu (C/C++, <code>strip</code> sonrası)	Java bayt kodu / DEX
Sınıf, metot ve alan adları	Kaybolur	Korunur (yansıma ve bağlama için gerekir)
Tipler	Kaybolur	Korunur (doğrulayıcı için gerekir)
Dize sabitleri	İkili dosyada durur	Sabit havuzunda (constant pool) düz metin durur
Kontrol akışı	Makine komutları, dolaylı atlamalar	Yapılandırılmış; kaynak koda çok yakın geri çevrilir
Geri çevirme sonucu	Okunması emek isteyen sözde C	Çoğu zaman derlenebilir Java kodu

Bu yüzden bir Java ya da Android uygulamasını tersine çevirmek için uzmanlık gerekmez. JDK'nin kendi aracı `javap`, bayt kodunu metin olarak gösterir; `jadx`, CFR, Fernflower gibi tersine derleyiciler (decompiler) ise okunabilir Java kaynağı üretir. Dokuzuncu haftada, gizlemeye karşı kullanılan bu tür araçları (sembolik yürütme, kalıp tanıma) ve bunlara karşı **dayanıklılık** ilkesini C/C++ tarafında derinlemesine işleyeceğiz ([Hafta 9, §10](#)).

`javap -c -p LisansDenetimi.class` (kısıtılmış)

```
private static final java.lang.String GECERLI_PIN;
public boolean pinDogru(java.lang.String);
Code:
  0: aload_1
  1: ldc      #7          // String 4729
  3: invokevirtual #9      // Method java/lang/String.equals:(Ljava/lang/Object;)Z
  6: ireturn
```

Bayt kodu satır satır: JVM bir yığın makinesidir

Yukarıdaki dört satırı hiç bayt kodu görmemiş biri için de okunur hale getirelim. JVM bir **yığın makinesidir**: her komut, küçük bir yığına (stack) değer **iter** (push) ya da yığından değer **çeker** (pop):

Satır	Komut	Ne yapar?	Yığının durumu (işlemden sonra)
0	<code>aload_1</code>	1 numaralı yerel değişkeni (metodun ilk parametresi, girilen PIN) yığına it	<code>[girilenPin]</code>
1	<code>ldc #7</code>	Sabit havuzundaki 7 numaralı girişi (dize <code>"4729"</code>) yığına it	<code>[girilenPin, "4729"]</code>
3	<code>invokevirtual #9</code>	Yığından iki değeri çek , <code>String.equals(...)</code> metodunu çağır, sonucu (true/false) yığına it	<code>[sonuc]</code>
6	<code>ireturn</code>	Yığındaki tamsayı/boole değeri metodun dönüş değeri olarak ver	<code>[]</code>

Bu dört satır, kaynaktaki `return girilenPin.equals("4729");` ifadesinin **birebir** karşılığıdır — hiçbir bilgi kaybolmamıştır. `// String 4729 yorumu, javap 'ın sabit havuzundaki 7 numaralı girişi okuyup göstermesidir`; tersine derleyiciler (jadx, CFR) bu dört satırı görüp doğrudan `return girilenPin.equals("4729");` **kaynak kodunu** üretir. C'de `strip` sonrası eşdeğer makine kodunda ne sabitin değeri ne de `equals` çağrısının hangi tür üzerinde olduğu bu kadar açık kalırdı (**dördüncü haftadaki** makine kodu örnekleriyle karşılaştırın).

Kaynak kod olmadan bile üç şey açıkça görünür: sabit (`4729`), anlamlı adlar (`GEZERLI_PIN`, `pinDogru`) ve mantık ("girdiyi şu sabitle karşılaştır"). Koda gömülü bir parola, API anahtarı ya da sunucu adresi, uygulamayı indiren herkese verilmiş demektir (MSC03-J, CWE-798).



Sık yapılan hata: 'kaynak kodu vermiyorum, güvendeyim' sanmak

Bir `.jar` ya da `.apk` dosyası dağıtmak, kaynak kodu vermemek anlamına gelmez: yukarıdaki tabloya göre bayt kodu neredeyse kaynak kodun kendisidir. "Kapalı kaynak" ile "tersine çevrilemez" birbirinden farklı şeylerdir.



Kural

İstemci tarafındaki bir Java/Kotlin uygulamasına giden her sabiti (anahtar, PIN, sunucu adresi, iş kuralı) "bunu gören biri kaynak kodu okuyormuş gibi düşünsün" varsayımıyla yazın. Bölüm 11–12, bu görünürlüğü **zorlaştıran** (ortadan kaldırmayan) araçları gösterecek.



İstemcide sır yoktur

Gizleme bu bölümde anlatılanları **zorlaştırrır**, ama bir sırrı istemci uygulamasında gerçekten saklamanın yolu değildir. Bir API anahtarı ya da parola istemcide durmak zorunda değilse durmamalıdır: sunucu tarafında tutulur, istemci kısa ömürlü bir belirteçle kimlik doğrular. İstemcide durmak zorunda olan anahtarlar için **3. haftadaki** kabuklar ve **11. haftadaki** whitebox kriptografi gerekir.

Demo 4 — Bayt kodunda ne görünür?

Demo 4 · `code/week-05/04-bytecode-decompile` · **CWE-798** · **MSC03-J**

Küçük bir "lisans/PIN denetimi" sınıfı derlenir ve `javap -c -p` ile incelenir. Kaynak kod olmadan sabit dizeler (sentetik PIN ve lisans anahtarı), `private` alan ve metot adları ve karşılaştırma mantığı görünür. Bu demo, gizleme ihtiyacının **motivasyonudur**; Demo 5 aynı sızıntıyı azaltır.

```
cd code\week-05\04-bytecode-decompile
.\demo.ps1
```

```
cd code/week-05/04-bytecode-decompile
sh demo.sh
```

11. ProGuard ve R8: küçültme, iyileştirme, gizleme

ProGuard, Java bayt kodunu işleyen açık kaynak bir araçtır; **R8** ise Google'ın Android için geliştirdiği ve Android derleme sisteminde varsayılan olarak gelen, ProGuard kurallarıyla uyumlu halefidir. İkisi de dört iş yapar:

-keep dengesi

doğrusu: yalnız dışarıdan erişilen giriş noktalarını keep et

ÇOK keep

reflection, serileştirme,
tüm public API korunur

→ adlar değişmez
→ gizleme ANLAMSIZ

AZ keep

yalnız dış giriş noktaları

→ adlar kısalır
→ ama reflection **KIRILIR**
(çalışma anı hatası)

Her -keep kuralı bir gizleme boşluğudur; en dar kuralı yazın ve test edin.

Aşama	Ne yapar?	Güvenlik katkısı
Küçültme (shrink)	Hiç kullanılmayan sınıf, metod ve alanları kaldırır (kütüphanelerdeki de dahil)	Saldırı yüzeyi ve incelenecek kod azalır
İyileştirme (optimize)	Satır içi alma, ölü kodu silme, sabit katlama	Kodun yapısı kaynaktan uzaklaşır
Gizleme (obfuscate)	Sınıf, metod ve alan adlarını <code>a</code> , <code>b</code> , <code>c</code> ... gibi kısa, anlamsız adlarla değiştirir	Anlamlı adlar kaybolur
Ön doğrulama (preverify)	Java ME / eski JVM'ler için doğrulama bilgisi ekler	—

Gizlemenin çıktısı, eski ve yeni adları eşleştiren bir **eşleme dosyasıdır** (`mapping.txt`):

mapping.txt (kısaltılmış, adlar sentetik)

```
com.ornek.odeme.KartYoneticisi -> com.ornek.a:
  android.content.Context baglam -> a
  boolean varsayilanKartiAyarla(java.lang.String) -> b
```



Eşleme dosyası bir sırdır

`mapping.txt`, gizlemeyi tamamen geri alır. Çökme raporlarını okuyabilmek için saklanması gerekir, ama uygulamayla birlikte **dağıtılmamalı**, herkese açık bir depoya konmamalı ve her sürüm için ayrı, erişimi sınırlı bir yerde tutulmalıdır.

-keep kuralları: neyin adı korunmalı?

Gizleyici, bir sınıfın ya da metodun **dışarıdan adıyla** çağrılıp çağrılmadığını bilemez. Aşağıdaki durumlarda adın korunması gerekir; aksi halde uygulama çalışma anında `ClassNotFoundException` ya da `NoSuchMethodException` verir:

- Uygulamanın giriş noktaları (Android'de manifest'te adı geçen etkinlikler ve servisler; masaüstünde `main`).
- Yansıma (reflection) ile adıyla çağrılan sınıf ve metodlar.
- JNI ile native koddan adıyla çağrılan metodlar ve `native` metodlar.
- Serileştirilen sınıfların alanları (JSON kütüphaneleri alan adlarını kullanır).
- Bir kütüphanenin dışa açık API'si.

Kural	Ne yapar?
<code>-keep class X { <methods>; }</code>	Sınıfı ve üyelerini hem tut hem adını koru
<code>-keepclassmembers class X { ... }</code>	Sınıf küçültülebilir ama belirtilen üyeler korunur (ör. serileştirme alanları)
<code>-keepnames / -keepclassmembernames</code>	Kullanılmayanlar silinebilir; kalanların adı korunur
<code>-keepclasseswithmembers</code>	Belirtilen üyelere sahip sınıfları korur (ör. <code>native</code> metodu olanlar)
<code>-dontobfuscate / -dontshrink / -donooptimize</code>	İlgili aşamayı kapatır

Kural yazmanın altın ilkesi **en dar kuralı** yazmaktır. `-keep class com.ornek.** { *; }` gibi geniş bir kural gizlemeyi fiilen kapatır; bu, sahada en sık görülen "gizleme var ama işe yaramıyor" durumudur.

İşlenmiş örnek: eksik `-keep` neden uygulamayı çökertir

Bir sınıfın adı yansımayla (`Class.forName("com.ornek.0deme")`) çağrılıyorsa ama `proguard-rules.pro` içinde bu sınıf için `-keep` yoksa, adım adım şu olur:

1. Derleme sırasında ProGuard/R8, `com.ornek.0deme` sınıfına **hiçbir statik çağrı görmez** — çünkü çağrı `Class.forName` içindeki bir **dize** olarak yazılıdır; gizleyici dizeleri sınıf grafiğinin parçası saymaz.
2. Gizleyici bu sınıfı "kullanılmıyor" sanıp **küçültme** aşamasında kaldırır ya da adını `a.b.c` 'ye değiştirir.
3. Uygulama çalışırken `Class.forName("com.ornek.0deme")` çağrılır; ama sürümdeki sınıfın adı artık farklıdır (ya da sınıf hiç yoktur).
4. Çalışma anında `ClassNotFoundException` (sınıf silinmişse) ya da `NoSuchMethodException` (metot adı değişmişse) fırlatılır — bu, **yalnızca sürüm (release) derlemesinde** görülür, çünkü hata ayıklama derlemesinde gizleme kapalıdır (Bölüm 13); geliştirici sorunu genelde ancak mağaza testinde fark eder.

Eksik `-keep`'in tipik çökme izi (illüstratif)

```
java.lang.ClassNotFoundException: com.ornek.0deme
    at java.lang.Class.forName(Class.java:...)
    at com.ornek.a.b.baslat(Unknown Source:1)
```



Sık yapılan hata: yansımayla çağrılan sınıfı unutmak

Statik çağrılar derleyici tarafından görülür ve otomatik olarak korunur; **yansıma, JNI ve serileştirme alanları görülmez**, çünkü bunlar sınıf grafiğinde değil, çalışma anında çözülen dizelerde yaşar. Bir kütüphane eklendiğinde (ör. JSON çözücü, bağımlılık enjeksiyonu) onun **hangi sınıfları adla çağırdığını** bilmeden gizleme açmak, üretimde rastgele çökmelere yol açar.

✓ Kural

Yeni bir kütüphane eklediğinizde önce onun resmi ProGuard/R8 kurallarını (çoğu kütüphane kendi `consumer-rules.pro` dosyasını taşır) ekleyin; sonra sürüm derlemesini **gerçekten çalıştırıp test edin**. "Derlendi" ile "çalışıyor" ProGuard/R8 açıkken aynı şey değildir.

Gelişmiş kurallar: sahadan bir yapılandırma

Sertifikasyondan geçmiş bir mobil ödeme kütüphanesinin gizleme yapılandırmasında şu kurallar ve gerekçeleri yer alır (kütüphanenin kendi kurallarının genelleştirilmiş hali):

Kural	Anlamı	Neden?
<code>-optimizations !code/simplification/arithmetic,!code/simplification/cast,!field/*,!class/merging/*</code>	Aritmetik ve tür dönüşümü sadeleştirme, alan iyileştirmeleri ve sınıf birleştirme dışındaki bütün iyileştirmeler	Bazı iyileştirmeler eski çalışma ortamlarında hataya yol açar; güvenli olanlar açık kalır
<code>-optimizationpasses 5</code>	İyileştirme beş tur tekrarlanır	Her tur, bir önceki turun açtığı yeni fırsatları kullanır; yapı kaynaktan daha çok uzaklaşır
<code>-allowaccessmodification</code>	Erişim belirteçleri genişletilebilir	Daha çok satır içi alma ve yeniden paketleme mümkün olur
<code>-repackageclasses</code>	Yeniden adlandırılan bütün sınıflar tek pakete taşınır	Paket yapısı (hangi sınıf hangi modülde) kaybolur
<code>-keepackagenames belirtilmez, -useuniqueclassmembenames</code>	Paket adları da gizlenir; aynı adlı üyeler tutarlı adlandırılır	Yapı ipucu kalmaz; ama çökme ayıklaması tutarlı olur
<code>-keepattributes Exceptions</code>	Yalnız istisna bildirimleri korunur	Kaynak dosya adı ve satır numarası (<code>SourceFile</code> , <code>LineNumberTable</code>) sürüme girmez
<code>-assumenosideeffects class android.util.Log { *; }</code>	Günlük çağrılar "yan etkisiz" sayılır ve silinir	Sürümde günlük dizeleri ve çağrılar kalmaz (4. haftadaki günlük kaldırmanın Java karşılığı)

🔧 Sahada nasıl uygulanır?

Aynı kılavuz önemli bir sorumluluk ayrımını da açıkça yazar: kütüphane kendi kodunu gizler ve küçültür, ama kütüphaneyi kullanan uygulamanın kodu üzerinde yetkisi yoktur; bu yüzden **üst uygulamanın da kendi gizleme ve tersine derlemeye karşı önlemlerini alması** kılavuzda gereksinim olarak devredilir. Kılavuzun "dış bileşenlerden beklenen güvenlik" bölümü tam olarak bu devirleri listeler.

ProGuard'ın yapmadıkları

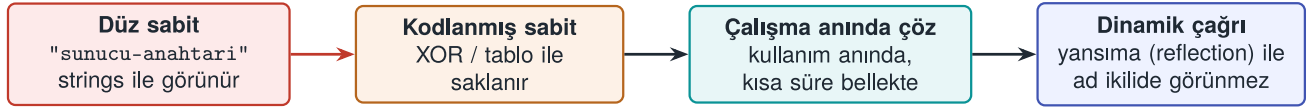
ProGuard ve R8 **adları** gizler, **dizeleri gizlemez**. `javap` çıktısındaki `// String 4729` satırı gizlemeden sonra da aynen durur. Kontrol akışını da ancak iyileştirmenin yan etkisi kadar değiştirir. Dize şifreleme, kontrol akışı gizleme ve kurcalamaya karşı denetimler için ticari araçlar (DexGuard, iXGuard gibi) ya da uygulamanın kendi yöntemleri gerekir (sonraki bölüm).

12. Dize gizleme ve dinamik yöntem çağrısı

Demo 4'teki sızıntının iki kaynağı vardı: **düz metin sabitler** ve **doğrudan çağrılar**. İki basit yöntem bunları azaltır.

Dize gizleme ve dinamik yöntem çağrısı

iki basit yöntem, iki farklı izi siler



Bu bir şifreleme değil, bir GECİKTİRME'dir: anahtar da ikilidedir, bulunabilir.

Statik dize gizleme

Dördüncü haftada **C tarafında sembol ve dize gizlemeyi** görmüştük; burada aynı fikri Java bayt kodunda uyguluyoruz. Hassas dize kaynaktaki düz metin olarak değil, **şifrelenmiş ya da karıştırılmış bir bayt dizisi** olarak durur ve kullanım anında çözülür. Sabit havuzunda dizinin kendisi yerine anlamsız baytlar görünür.

- Basit bir XOR karıştırması `javap` ve `strings` taramasını durdurur; ama çözme fonksiyonu ve anahtarı da uygulamanın içindedir. Bu bir **engel**, kilit değildir.
- Sahada dizeler derleme öncesinde bir araçla **AES ile şifrelenir** ve derleme yapılandırmasına şifreli sabitler olarak gömülür; çözme işi native tarafta, ek denetimlerle korunan tek bir fonksiyona bırakılır ve çözülen dize iş bitince bellekten silinir. Böylece Java tarafında ne dizinin kendisi ne de çözme anahtarı bulunur.
- Çözülen dize kullanım anında bellekte açıktır; bu yüzden dize gizleme, çalışma anı korumasıyla (RASP, **6. hafta**) birlikte anlam kazanır.

İşlenmiş örnek: XOR gizlemeyi elle çözmek

Demo 5'in `GizliSabit.java` dosyası, `"sunucu-anahtari-9F3A"` sentetik dizesini XOR anahtarı `0x5A` (ondalık 90) ile karıştırıp bayt dizisi olarak saklar. Kaynaktaki dizinin ilk üç baytını (41, 47, 52) elle çözelim; XOR'un tanımı gereği aynı anahtarla **iki kez** XOR'lamak orijinal değeri geri verir ($x \oplus k \oplus k = x$) — bu yüzden şifreleme ve çözme **aynı** işlemdir:

1. bayt: $41 \oplus 90 = 's' (0x73)$

```

41 = 0010 1001
90 = 0101 1010
----- (bit bit XOR: aynıysa 0, farklıysa 1)
115 = 0111 0011 = 0x73 = 's'

```

2. bayt: $47 \oplus 90 = 'u' (0x75)$

```

47 = 0010 1111
90 = 0101 1010
-----
117 = 0111 0101 = 0x75 = 'u'

```

3. bayt: $52 \oplus 90 = 'n' (0x6E)$

```

52 = 0011 0100
90 = 0101 1010
-----
110 = 0110 1110 = 0x6E = 'n'

```

Kalan 17 bayt aynı işlemle çözülür ve sırasıyla u, c, u, -, a, n, a, h, t, a, r, i, -, 9, F, 3, A karakterlerini verir; birleştirince tam olarak kaynaktaki "sunucu-anahtari-9F3A" dizesi ortaya çıkar. `javap -c -p` bu dizeyi **değil**, yalnızca 20 anlamsız bayt değeri ve ANAHTAR sabitini gösterir; `strings` taraması da düz metni bulamaz. Ama anahtar (`0x5A`) ve çözme döngüsü **aynı sınıfın içindedir** — bu yüzden aşağıdaki uyarı geçerlidir.

⚡ Sık yapılan hata: XOR gizlemeyi 'şifreleme' sanmak

Tek baytlık bir XOR anahtarı, yalnız 256 olasılıktan biridir; anahtar ve çözme kodu aynı `.class` dosyasında bulunduğu için bir tersine derleyici ikisini de görüp aynı hesabı **otomatik** yapabilir. Bu, kriptografik bir şifreleme değil, `javap / strings` gibi **düz metin tarayan** araçları yavaşlatan bir engeldir.

✓ Kural

Dize gizleme "sırrı saklar" demek değildir; "sabit havuzunda düz metin bırakmaz" demektir. Gerçekten hassas bir değer (üretim sunucusunun adresi, gerçek bir API anahtarı) hiçbir zaman istemci koduna gömülmemeli; sunucudan çalışma anında alınmalı ya da [11. haftadaki](#) whitebox kriptografi gibi özel yöntemlerle korunmalıdır.

Dinamik yöntem çağırısı (yansıma)

Bir metodu doğrudan çağırarak (`nesne.gizliIslem()`), bayt kodunda o metoda giden açık bir bağlantı (`invokevirtual`) bırakır; tersine derleyici "bu fonksiyon şuradan çağırılıyor" grafiğini kolayca çıkarır. **Yansıma** ile çağrıda metod adı çalışma anında üretilir (ve bu ad da gizlenebilir):

Doğrudan ve dinamik çağrı

```

sonuc = Hesap.gizliIslem(girdi); // bayt kodunda açık bağlantı

Method m = Hesap.class.getDeclaredMethod(adiCoz(ADI_BAYTLARI), String.class);
m.setAccessible(true);
sonuc = (String) m.invoke(null, girdi); // statik çağrı grafiği kırılır

```

Sınırları dürüstçe söyleyelim: metot hâlâ sınıfta tanımlıdır ve `javap -p` onu listeler; yansıma yalnız **çağrı bağlantısını** gizler. Asıl güç, ProGuard'ın metodu yeniden adlandırmasıyla birleştiğinde ortaya çıkar (bu durumda yansımayla çağrılan metot için `-keep` kuralı gerekir; kural, adı çalışma anında üretilen yeni adla eşleşecek biçimde yazılır). Yansıma ayrıca yavaşır ve derleme anındaki tip denetimini kaybettirir; yalnız gerçekten hassas birkaç çağrıda kullanılır.

Sahada nasıl uygulanır?

Java ile native kütüphane arasındaki JNI arayüzünde metot adları anlamsızlaştırılır: `native byte[] ced(...)` gibi iki üç harflik adlar, hangi metodun ne yaptığını gizler (JNI adları native tarafta sembol olarak görüldüğü için bu, iki tarafı birden korur). Cihaz parmak izi gibi bağlama değerleri ise Java tarafında **iki farklı özetle iki farklı yerde** (biri yerel veritabanında, biri uygulama tercih dosyasında) saklanır ve bir arka plan iş parçacığı ikisini düzenli olarak karşılaştırır; tutarsızlık ya da root algılaması kartların silinmesini tetikler. Tek bir yerdeki değeri değiştirmek bu yüzden yetmez.

Demo 5 — Dize gizleme, yansıma ve ProGuard

Demo 5 · `code/week-05/05-gizleme` · Tarif 12.11 (kavram), 12.9 (dolaylı çağrı, kavram)

Üç savunma sırayla gösterilir: **(A)** aynı gizli dize düz metin yerine XOR ile karıştırılmış bir bayt dizisi olarak durur ve `javap` çıktısında görünmez; **(B)** metot adı çalışma anında üretilip yansımayla çağrılır ve bayt kodunda doğrudan çağrı kalmaz; **(C)** isteğe bağlı olarak aynı jar ProGuard ile küçültülür ve gizlenir: `-keep` ile korunan giriş noktası kalır, kullanılmayan özel üye kaldırılır, kalanlar kısa adlarla yeniden adlandırılır. ProGuard `hazır1a` betiğiyle demo klasörüne indirilir.

```
cd code\week-05\05-gizleme
.\demo.ps1
```

```
cd code/week-05/05-gizleme
sh demo.sh
```

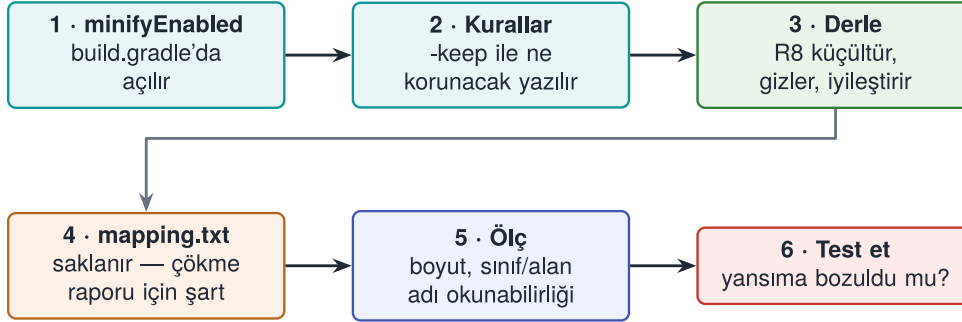
Değerlendirici nasıl test eder?

Uygulama paketini (JAR/APK) bir tersine derleyiciyle açar ve dört soru sorar: anlamlı sınıf ve metot adları kaldı mı, hassas dizeler (sunucu adresi, anahtar, dosya adı) düz metin olarak görünüyor mu, günlük çağrıları sürümde duruyor mu, eşleme dosyası paketin içinde ya da herkese açık bir yerde mi? Ardından güvenlik denetimlerinin (root algılama, sabitleme) geri derlenmiş koddaki yerini bulmanın ne kadar sürdüğünü ölçer.

13. Android'de R8, gizlemenin etkisini ölçmek ve Java için statik analiz

Android'de R8 ve gizlemenin etkisini ölçmek

ölçmediğiniz gizleme bir iddiadır



mapping.txt kaybolursa üretimdeki çökme raporlarını bir daha çözemezsiniz.

Android derleme hattında R8

Android uygulamalarında R8, Gradle yapılandırmasındaki birkaç satırla açılır. Sürüm derlemesinde açık olması, mobil güvenlik standartlarının (OWASP MASVS-RESILIENCE) temel beklentisidir:

app/build.gradle — sürüm derlemesinde küçültme ve gizleme

```

android {
    buildTypes {
        release {
            minifyEnabled true           // R8: küçültme + iyileştirme + gizleme
            shrinkResources true         // kullanılmayan kaynakları (resim, düzen) da kaldır
            proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'),
                'proguard-rules.pro'
        }
    }
}
  
```

proguard-android-optimize.txt, Android için makul varsayılanları içerir; uygulamaya özgü -keep kuralları proguard-rules.pro dosyasına yazılır. JNI ile native koddan çağrılan metotlar ve native metotlar için tipik kural:

proguard-rules.pro — JNI

```

-keepclasseswithmembernames class * {
    native <methods>;
}
  
```

⚠ Hata ayıklama derlemesinde gizleme yoktur

Android Studio'da çalıştırılan hata ayıklama (debug) derlemesi varsayılan olarak **gizlenmez** ve ayrıca "debuggable" işaretlidir. Dağıtılan paketin **sürüm** derlemesi olduğunu ve `debuggable` bayrağının kapalı olduğunu doğrulamak, değerlendiricinin ilk kontrollerinden biridir (6. haftada hata ayıklayıcı algılama).

Çökme raporlarını okumak: retrace

Gizlenmiş bir uygulamanın çökme raporu `a.b.c(Unknown Source)` gibi satırlar içerir. Eşleme dosyası, bu raporu geri çevirmek için kullanılır (`retrace` aracı ya da çökme raporlama hizmetlerine eşleme dosyasının **güvenli biçimde** yüklenmesi). Bu yüzden eşleme dosyası silinmez; ama yukarıda söylediğimiz gibi uygulamayla birlikte de dağıtılmaz. - `keepattributes SourceFile,LineNumberTable` eklemek raporları okunaklı yapar ama kaynak dosya adlarını ve satır numaralarını pakete koyar; bu da bir ödünleşimdir ve kayda geçirilir.

Gizlemenin etkisini ölçmek

"Gizleme açık" demek yetmez; gerçekten ne değiştiğini göstermek gerekir. Basit ve tekrarlanabilir ölçütler:

Ölçüt	Nasıl ölçülür?	Beklenen değişim
Sınıf ve metot sayısı	<code>javap</code> / tersine derleyici çıktısı	Küçültmeyle azalır
Anlamlı ad oranı	Uzunluğu 3'ten büyük sınıf/metot adlarının oranı	Gizlemeyle çok düşer (korunan giriş noktaları dışında)
Düz metin hassas dize sayısı	<code>strings</code> / sabit havuzu taraması, bilinen hassas dizelerin listesiyle	Dize gizlemeyle sıfıra iner
Paket boyutu	Dosya boyutu	Küçültmeyle azalır (gizleme tek başına pek değiştirmez)
Günlük çağırısı sayısı	Geri derlenmiş kodda <code>Log.</code> araması	Sürümde sıfır
Bir güvenlik denetimini bulma süresi	İnsan deneyi	Artar; 9. haftada ölçme yöntemleri

Bu tabloyu gizleme öncesi ve sonrası için doldurup güvenlik kılavuzuna koymak, "gizleme yapıldı" iddiasını kanıta dönüştürür.

İşlenmiş örnek: ölçümü Demo 5 üzerinde adım adım yapmak

Bu tablonun ilk iki satırını Demo 5'in ürettiği jar dosyaları üzerinde gerçekten çalıştıralım (demo `hazirla` betiğiyle ProGuard'ı indirdiyseniz):

1) Gizleme öncesi: anlamlı ad sayısını say

```
cd code/week-05/05-gizleme/cikti
javap -p -classpath oncesi.jar 'GizliSabit' | grep -c "GizliSabit\|coz\|GIZLI\|ANAHTAR"
# Cikti: kaynakteki adların hepsi (sınıf, alan, metot) birebir görünür
```

2) Gizleme sonrası: aynı arama

```
javap -p -classpath sonrasi.jar 'a' | grep -c "GizliSabit\|coz\|GIZLI\|ANAHTAR"
# Beklenen: 0 (ya da yalnızca -keep ile korunan giris noktasının adı)
```

3) Paket boyutu

```
ls -la oncesi.jar sonrasi.jar
# Kucultme sinif sayisina bagli olarak boyutu azaltir; tek basina gizleme boyutu pek degistirmez
```

Bu üç komut, "gizleme çalıştı" iddiasını **ölçülebilir bir sayıya** dönüştürür: birinci komutun çıktısı sıfırdan büyükse (anlamalı ad hâlâ bulunuyorsa), ya `-keep` kuralı fazla geniş yazılmıştır ya da gizleme aşaması hiç çalışmamıştır — ikisi de Bölüm 11'deki "sahada en sık görülen hata" ile aynı belirtidir.

Java için statik analiz

Dördüncü haftada C/C++ için gördüğümüz statik analiz katmanlarının Java karşılıkları:

Araç	Ne bulur?
Derleyici uyarıları (<code>javac -Xlint:all</code>)	Güvensiz dönüşüm, kullanımdan kalkmış API
SpotBugs + Find Security Bugs eklentisi	SQL/komut/yol enjeksiyonu kalıpları, zayıf kriptoloji, güvensiz seri durumdan çıkarma, gömülü parola
Semgrep, CodeQL	Kaynaktan hedefe veri akışı (kullanıcı girdisi → <code>executeQuery</code>)
Android Lint	Android'e özgü güvenlik uyarıları (dışa açık bileşenler, güvensiz ağ yapılandırması)
OWASP Dependency-Check (Maven/Gradle eklentisi)	Zafiyetli bağımlılıklar (14. bölüm)

Maven projesinde SpotBugs + Find Security Bugs

```
mvn com.github.spotbugs:spotbugs-maven-plugin:check
# pom.xml'de eklenti yapılandırmasına findseccbugs-plugin eklenir
```

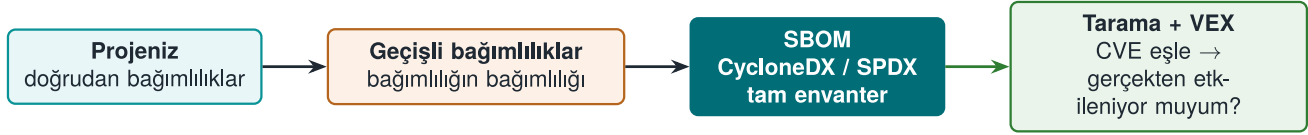
Bu araçlar, 4. haftadaki güvenli derleme hattına (CI) aynı biçimde eklenir: yeni bir yüksek önem dereceli bulgu, birleştirmeyi durdurur.

14. Bağımlılık güvenliği ve yazılım malzeme listesi (SBOM)

Modern bir uygulamanın kodunun büyük kısmı, onu yazan ekibe ait değildir. Tipik bir Java ya da JavaScript uygulamasında doğrudan eklenen birkaç düzine kütüphane, onların bağımlılıklarıyla birlikte yüzlerce bileşene ulaşır. Bu bileşenlerden birindeki bir zafiyet, uygulamanın zafiyetidir.

Tedarik zinciri: SBOM neden gerekli?

Log4Shell'de asıl sorun "bende var mı?" sorusuydu



Sürümleri sabitleyin: yeniden üretilebilir derleme + beklenmedik güncellemeye karşı koruma.



Log4Shell (Aralık 2021)

Java dünyasının en yaygın günlük kütüphanelerinden Log4j 2'de, günlüğe yazılan bir dizinin içindeki özel bir ifade, kütüphanenin uzaktaki bir adresten sınıf yükleyip çalıştırmasına yol açıyordu (CVE-2021-44228, CVSS 10.0). Günlüğe kullanıcı verisi yazan her uygulama etkilendi. Asıl kriz yamayı uygulamak değil, **hangi sistemde hangi sürümün bulunduğunu bilmek** oldu: çoğu kurum, Log4j'nin başka bir kütüphanenin içine gömülü olarak hangi uygulamalarında bulunduğunu günlerce aradı. Olay, **2. haftadaki** günlük enjeksiyonu ve bu bölümdeki bağımlılık güvenliği konularını birleştirir.

SBOM nedir?

Yazılım malzeme listesi (Software Bill of Materials, SBOM), bir yazılımın içindeki bütün bileşenlerin makine tarafından okunabilir listesidir: gıda paketinin üzerindeki içindekiler listesi gibi. Her bileşen için en az şunlar yazılır:

Alan	Örnek	Neden?
Ad ve sürüm	log4j-core 2.14.1	Zafiyet veritabanlarıyla eşleştirmek
purl (paket URL'si)	pkg:maven/org.apache.logging.log4j/log4j-core@2.14.1	Ekosistemden bağımsız, tek biçimli kimlik
Özet değeri	SHA-256	Dosyanın gerçekten o bileşen olduğunu doğrulamak
Lisans	Apache-2.0	Lisans uyumluluğu
Tedarikçi, bağımlılık ilişkileri	kimin, neye bağlı	Geçişli bağımlılıkları izlemek

İki yaygın standart vardır: OWASP'ın **CycloneDX**'i (güvenlik odaklı) ve Linux Vakfı'nın **SPDX**'i (lisans odaklı başladı, ISO/IEC 5962 standardı). ABD'nin 2021 tarihli siber güvenlik kararnamesi ve Avrupa Birliği'nin Siber Dayanıklılık Yasası (Cyber Resilience Act), yazılım tedarikçilerinden SBOM istemeye başlamıştır.

CycloneDX 1.5 — tek bileşenlik kesit (değerler sentetik)

```
{
  "bomFormat": "CycloneDX",
  "specVersion": "1.5",
  "components": [ {
    "type": "library",
    "name": "ornek-json",
    "version": "1.2.0",
    "purl": "pkg:maven/com.ornek/ornek-json@1.2.0",
    "hashes": [ { "alg": "SHA-256", "content": "9f86d081884c7d65..." } ]
  } ]
}
```

SBOM'dan zafiyet listesine

SBOM tek başına bir envanterdir; değerini **yazılım bileşen analizi** (SCA) araçlarıyla birleşince kazanır. Bu araçlar SBOM'daki her bileşeni zafiyet veritabanlarıyla (NVD, GitHub Advisory Database, OSV) eşleştirir:

Araç	Tür
OWASP Dependency-Check	Proje taraması, açık kaynak
OWASP Dependency-Track	SBOM'ları toplayıp sürekli izleyen platform, açık kaynak
GitHub Dependabot, <code>npm audit</code> , <code>pip-audit</code> , OSV-Scanner	Depo ve ekosistem düzeyinde uyarı
CycloneDX eklentileri (Maven, Gradle, npm, pip)	SBOM üretimi

Bir eşleşme her zaman "etkileniyoruz" demek değildir: zafiyetli fonksiyon hiç çağırılmıyor olabilir. Bu bilgiyi makine tarafından okunabilir biçimde yayımlamak için **VEX** (Vulnerability Exploitability eXchange) belgeleri kullanılır: "bu bileşende CVE-X var, ama ürünümüz etkilenmiyor, çünkü ...".

Bağımlılık hijyeni: kontrol listesi

- ✓ **Sürümleri sabitle:** kilit dosyası (`package-lock.json`, `gradle.lockfile`, `poetry.lock`) depoda olsun; "en son sürüm" gibi kayan sürümler kullanılmasın.
- ✓ **Özetleri doğrula:** Maven/Gradle bağımlılık doğrulama, `pip --require-hashes`; indirilen dosya beklenen dosya mı?
- ✓ **Kullanılmayana kaldır:** sınıf yolundaki her kütüphane bir saldırı yüzeyidir (seri durumdan çıkarma zincirleri kullanılmayan kütüphanelerden de kurulabilir).
- ✓ **Sürekli izle:** SBOM'u her sürümde üret, izleme platformuna gönder; yeni bir CVE yayımlandığında hangi ürünlerin etkilendiğini dakikalar içinde bil.
- ✓ **Kaynağı denetle:** ad benzetme (typosquatting: `requeusts` gibi) ve bağımlılık karışıklığı (iç paket adının herkese açık depoda sahte bir paketle ele geçirilmesi) saldırılarına karşı paket kaynağını sabitle.
- ✓ **Derleme bütünlüğü:** 2. haftadaki tedarik zinciri olayları ve SLSA çerçevesi: derlemenin kendisi de kanıtlanabilir olmalı.

**Sık yapılan hata: SBOM'u bir kere üretilip unutmak**

SBOM, üretildiği **andaki** bağımlılık listesidir. Bir sürüm sonra yeni bir kütüphane eklenir ya da bir sürüm yükseltirise, eski SBOM artık **yanlış** bilgi verir — üstelik yanlış olduğu hiçbir yerde yazmaz, sadece güncel değildir. Log4Shell'de asıl zaman kaybı, "SBOM'umuz vardı ama üç ay önceki sürüme aitti" durumundan kaynaklandı.

**Kural**

SBOM, derleme hattının (CI) her çalışmasında **otomatik** üretilmeli ve bir SCA aracına gönderilmelidir; elle bir kez hazırlanıp belgeye eklenen bir SBOM, üretildiği gün doğrudur, bir sonraki bağımlılık güncellemesinde değil.

İşlenmiş örnek: Demo 6'nın çıktısını satır satır okumak

Demo 6 (`sbom.py`), dört sentetik kütüphaneden küçük jar dosyaları üretir, her biri için CycloneDX bileşeni yazar ve küçük bir "bilinen zafiyetli sürümler" sözlüğüyle eşleştirir. Gerçek çıktı (bileşen adları ve sürümleri kaynak koddaki sabit listeden):

ADIM 1 — SBOM üretildi (4 bileşen)

ad	sürüm	SHA-256 (ilk 16)
gunluk-cekirdek	2.14.0	<jar baytlarından hesaplanır>...
json-arac	2.5.1	<jar baytlarından hesaplanır>...
kayit-kutuphanesi	1.2.0	<jar baytlarından hesaplanır>...
sifreleme-yardimci	3.0.4	<jar baytlarından hesaplanır>...

ADIM 2 — purl örnekleri

```
pkg:maven/ornek.grup/gunluk-cekirdek@2.14.0
pkg:maven/ornek.grup/json-arac@2.5.1
pkg:maven/ornek.grup/kayit-kutuphanesi@1.2.0
pkg:maven/ornek.grup/sifreleme-yardimci@3.0.4
```

ADIM 3 — Bilinen (sentetik) zafiyetli sürümlerle eşleştirme

```
[UYARI] gunluk-cekirdek 2.14.0 (CEN429-2026-0001)
        Bicimli mesajda uzaktan kod calistirma (sentetik ornek).
        Cozum: >= 2.17.1 surumune yukselt.
[UYARI] json-arac 2.5.1 (CEN429-2026-0002)
        Guvensiz seri durumdan cikarma (sentetik ornek).
        Cozum: >= 2.6.0 surumune yukselt.
```

SHA-256 değerleri her çalıştırmada **farklı** çıkar (üretilen jar'ın ZIP meta verisi zaman damgası içerir); bu kasıtlıdır ve iyi bir tedarik zinciri dersidir — özet, dosyanın **tam olarak hangi bayt dizisi** olduğunu doğrular, yalnız "ad ve sürüm aynı" demek yeterli değildir. `kayit-kutuphanesi` ve `sifreleme-yardimci`, sentetik zafiyet listesinde bulunmadığı için **uyarı almaz** — bir SBOM'un boş kalan kısmı da bilgidir: "bu bileşenler bilinen listede yok" demektir, "güvenli" demek değildir.

Demo 6 — SBOM üretmek ve zafiyetle eşleştirmek

Demo 6 · `code/week-05/06-sbom` · CWE-1104

Demo, kendi ürettiği **sentetik** jar dosyalarından (`cikti/lib/`) bir CycloneDX 1.5 JSON SBOM'u üretir: her bileşen için ad, sürüm, purl ve SHA-256. Ardından küçük, **sentetik** bir "bilinen zafiyetli sürümler" listesiyle eşleştirir ve etkilenen iki bileşen için uyarı verir. İnternete bağlanmaz.

```
cd code\week-05\06-sbom
.\demo.ps1
```

```
cd code/week-05/06-sbom
sh demo.sh
```

Sahada nasıl uygulanır?

Sertifikasyon belgelerinde üçüncü taraf bileşenler ayrı bir tablo halinde, sürümleri ve **neden kullanıldıkları** ile listelenir: ödeme şemalarının SDK'ları, platformun bütünlük doğrulama API'si, bildirim servisi, whitebox kütüphanesi, sayı teorisi kütüphanesi. Değerlendirici bu listeyi bir SBOM gibi okur: her bileşenin bilinen bir zafiyeti var mı, güncel mi, gerçekten gerekli mi? Listede olmayan bir bileşenin pakette bulunması ayrıca bir bulgudur.

15. Dönem projesi: bu hafta

Projeniz C/C++ olsa bile bu haftanın iki konusu doğrudan projenize uygulanır:

- ✓ **SBOM:** Projenizin bütün bağımlılıklarını (kütüphaneler, derleme araçları, CMake ile indirilen bileşenler) CycloneDX biçiminde listeleyin. vcpkg ya da Conan kullanıyorsanız SBOM üretim desteklerini deneyin; yoksa elle yazın. Her bileşeni OSV veritabanında arayın ve sonucu kılavuza ekleyin.
- ✓ **Girdi doğrulama tablosu:** Projenizin dışarıdan veri aldığı her noktayı (dosya, komut satırı, ağ, yapılandırma) listeleyin; her biri için beklenen biçimi, uzunluk sınırını ve doğrulamanın yapıldığı fonksiyonu yazın.
- ✓ Projeniz bir dosya yolu, bir SQL sorgusu ya da bir dış program çağrısı içeriyorsa, bu haftanın kalıplarıyla (kanonikleştirme + kök, parametrelili sorgu, argüman listesi) yeniden gözden geçirin.
- ✓ Projenizde bir betik ya da Java bileşeni varsa, gizleme gereksinimini ve `-keep` kurallarını S9 bölümüne ekleyin.

16. Kendi başına çalış

Alıştırma 1 — Kolay: Parametrelili sorguya çevirin



Demo 1'deki hatalı sorguyu kendiniz parametrelili hale getirin. Sonra bir `ORDER BY` sütununu kullanıcıdan alan bir sıralama özelliği ekleyin ve bunu beyaz listeyle güvenli yapın. Neden `?` ile yapılamadığını açıklayın.

? Alıştırma 2 — Kolay: Hata iletisinin sızdırdıkları

Demo 1'de hatalı sorguya sözdizimi bozuk bir girdi verin ve veritabanının hata iletisini inceleyin. İletiden tablo ve sütun adları çıkarılabilir mi? Kullanıcıya genel ileti, günlüğe ayrıntı yazacak şekilde düzeltin.

? Alıştırma 3 — Orta: Argüman enjeksiyonu

Demo 2'deki güvenli sürüme `-` ile başlayan bir girdi verin. İzin listesi bunu reddediyor mu? Reddetmiyorsa, çağrılan programın bu girdiyi seçenek olarak yorumlayıp yorumlamadığını gözlemleyin ve `--` ayırıcısını ekleyin.

? Alıştırma 4 — Orta: Yol geçişi varyantları

Demo 3'ün güvenli sürümüne şu istekleri gönderin: `..%2fgizli.txt`, `....//gizli.txt`, mutlak yol, Windows'ta `..\gizli.txt` ve sürücü harfli yol, kök içinde **sembolik bağlantıyla** kök dışını gösteren bir dosya. Hangisi hangi denetimde reddediliyor?

? Alıştırma 5 — Orta: Filtresiz ve filtreli seri durumdan çıkarma

Demo 7'deki filtre desenini `!*` olmadan yazın ve beklenmeyen sınıfın bu kez kabul edildiğini gösterin. Sonra `maxbytes` ve `maxdepth` sınırlarını çok küçük değerlere çekip meşru bir nesnenin reddedildiği noktayı bulun.

? Alıştırma 6 — Orta: Tersine derleyiciyle bakın

Demo 4 ve Demo 5'in jar dosyalarını `javap -c -p` dışında bir tersine derleyiciyle (ör. CFR ya da jadx) açın. Gizleme öncesi ve sonrası okunabilirliği karşılaştırın: hangi bilgi hâlâ görünüyor?

? Alıştırma 7 — Zor: -keep kuralını daraltın

Demo 5'in `proguard.pro` dosyasındaki `-keep` kuralını daha geniş (`com.**`) ve daha dar yazıp sonuçları karşılaştırın. Uygulamanın hâlâ çalıştığı **en dar** kural nedir?

? Alıştırma 8 — Zor: Gerçek bir SBOM

Maven ya da Gradle kullanan herhangi bir açık kaynak Java projesi için CycloneDX eklentisiyle SBOM üretin ve OWASP Dependency-Check ya da OSV-Scanner ile tarayın. Kaç bileşen, kaç geçişli, kaç zafiyet eşleşmesi çıktı? Eşleşmelerden birinin projesi gerçekten etkileyip etkilemediğini araştırın.

17. Kendini sinama

? 1. Java hangi hata sınıflarını dil düzeyinde ortadan kaldırır, hangilerini kaldırmaz?



Arabellek taşması, serbest bırakılmış bellek, ilklendirilmemiş değişken gibi bellek hatalarını kaldırır. Enjeksiyon, güvensiz seri durumdan çıkarma, koda gömülü sırlar, tersine mühendislik, zafiyetli bağımlılıklar ve mantık hatalarını kaldırmaz.

? 2. Bütün enjeksiyon türlerinin ortak kökü ve ortak düzeltilmesi nedir?



Veri ile komutun aynı kanaldan bir yorumlayıcıya verilmesi. Düzeltme: komutu sabit tutup veriyi ayrı bir kanaldan (parametre, argüman) vermek.

? 3. `ORDER BY` sütunu neden ? parametresiyle verilemez? Ne yapılır?



? yalnız değer yerine konabilir, tanımlayıcı (sütun, tablo adı) yerine konamaz. Beyaz listeye eşleme yapılır.

? 4. `ProcessBuilder` kullanmak komut enjeksiyonunu neden önler? Tek başına her şeyi çözer mi?



Kabuk çalışmaz; girdi tek bir argüman olarak geçer. Çözmez: çağrılan program argümanı seçenek olarak yorumlayabilir (argüman enjeksiyonu); izin listesi ve `--` gerekir.

? 5. Yol geçişinde neden önce kanonikleştirilir, sonra denetlenir?



Aynı yolun birçok yazılışı (`./` , kodlanmış biçimler, sembolik bağlantılar) vardır; ham dize denetimi atlatılır. Kanonik yol tek ve kesindir.

? 6. Güvensiz seri durumdan çıkarmada saldırganın elindeki yetki nedir?



Hangi sınıfların oluşturulacağını seçme yetkisi. Sınıf yolundaki sınıflar zincirlenerek kod çalıştırmaya kadar gidebilir.

? 7. `ObjectInputFilter` deseninin sonundaki `!*` ne işe yarar?



Listede olmayan her şeyi reddeder; varsayılan-reddet ilkesi. Olmazsa filtre bir kara listeye dönüşür.

? 8. Java bayt kodu neden makine kodundan daha kolay geri çevrilir?



Sınıf, metot ve alan adlarını, tipleri ve dizeleri korur; kontrol akışı yapılandırılmıştır.

? 9. ProGuard'ın dört aşaması nedir? Hangisi dizeleri gizler?

Küçültme, iyileştirme, gizleme (ad değiştirme), ön doğrulama. Hiçbiri dizeleri gizlemez.

? 10. `mapping.txt` neden bir sırdır?

Gizlemeyi tamamen geri alır; eski ve yeni adları eşler. Dağıtılmamalı, erişimi sınırlı saklanmalıdır.

? 11. `-assumenosideeffects class android.util.Log { *; }` ne işe yarar?

Günlük çağrılarını yan etkisiz sayarak sürüm derlemesinden siler; günlük dizeleri ve çağrıları ikili dosyada kalmaz.

? 12. SBOM'da purl ve özet değeri neden bulunur? VEX ne işe yarar?

purl bileşeni ekosistemden bağımsız tek biçimde tanımlar; özet dosyanın gerçekten o bileşen olduğunu doğrular. VEX, bir bileşendeki zafiyetin ürünü gerçekten etkileyip etkilemediğini bildirir.

? 13. SQL ayrıştırıcısı, girdideki bir `'` karakteriyle karşılaştığında hangi duruma geçer? Bu durum neden sorgunun yapısını değiştirir?

"Dize sabiti başladı/bitti" durumuna geçer. Girdideki `'`, açık olan bir dize sabitini erken kapatır; ondan sonraki metin artık dize içeriği değil, normal SQL söz dizimi (anahtar sözcük, operatör) olarak okunur.

? 14. Parametrelili sorguda `hazırlama` (prepare) ve `bağlama` (bind) aşamaları neden ayrıdır? Bu ayrım enjeksiyonu nasıl engeller?

Hazırlama aşamasında yalnız sabit sorgu metni ayrıştırılıp bir plana dönüştürülür; kullanıcı verisi henüz yoktur. Bağlama aşamasında değerler ayrı bir kanaldan, zaten ayrıştırılmış plana yerleştirilir ve bir daha hiç ayrıştırıcıdan geçmez; bu yüzden içeriği ne olursa olsun sorgunun yapısını değiştiremez.

? 15. Kabuk, `echo Merhaba ayse; echo SIZDI` dizesini kaç komut olarak çalıştırır? Neden?

İki komut. Kabuk, aldığı metni önce `;` gibi komut ayırıcılara göre ayrı komutlara böler; bu, kabuğun kendi söz dizimi kuralıdır ve Java'nın "bu tek bir kullanıcı adıydı" bilgisiyle ilgisi yoktur.

? 16. `K0K.resolve("../etc/passwd")` ifadesini yığın (stack) algoritmasıyla adım adım çözün: sonuç nedir?

Her `..` segmenti yığından bir önceki segmenti pop'lar, her normal segment yığına push'lanır. İki `..`, kökün son iki segmentini (`srv`, `veri`) çıkarır; kalan `etc` ve `passwd` push'lanır. Sonuç `/etc/passwd` — kökün tamamen dışında.

? 17. `ObjectInputStream`, akıştaki hangi bilgiyi okuyarak hangi sınıfın oluşturulacağına karar verir? Bu bilgi kim tarafından belirlenir? ✓

Akıştaki sınıf tanımlayıcısı bloğunda düz metin olarak yazılan **tam sınıf adını** okur ve bu adı `Class.forName` benzeri bir çağrıyla yükler. Bu adı akışı **gönderen taraf** belirler; akış güvenilmezse saldırgan belirler.

? 18. XXE'de bir `SYSTEM` varlığının içeriği ne zaman "genişletilir"? Uygulamanın girdi doğrulaması bunu neden yakalayamaz? ✓

Ayrıştırma sırasında, belge gövdesi işlenirken; uygulamanın kendi mantığı devreye girmeden önce. Girdi doğrulama içeriğe bakar, ama varlık genişletme belgenin **yapısal** bir özelliğidir; uygulamanın hiçbir denetiminden geçmeden ayrıştırıcı düzeyinde gerçekleşir.

? 19. `^(a+)+$` deseninde girdi 4 tane `a` içeriyorsa, motorun iç grubu bölümleme sayısı kaçtır? Genel formül nedir? ✓

$8 (2^{(4-1)} = 2^3 = 8)$. Genel formül: n tane `a` için $2^{(n-1)}$ farklı bölümleme (kompozisyon) vardır.

? 20. `^(a+)+$` deseninde girdinin sonuna eşleşmeyi bozan tek bir karakter eklemek ("`aaaa!`" gibi) neden ReDoS'u tetikler? ✓

Girdi hâlâ kısa olsa da (n küçük), eşleşme `$` ile asla sağlanamadığı için motor mümkün olan **bütün** $2^{(n-1)}$ bölümlmeyi dener ve hepsi başarısız olduktan sonra "eşleşme yok" sonucuna varır; n büyüdükçe bu sayı üstel büyür.

? 21. Java bayt kodunda `ldc` ve `invokevirtual` komutları JVM yığnında ne yapar? ✓

`ldc`, sabit havuzundaki bir değeri (sayı, dize) yığna iter. `invokevirtual`, yığından gerekli işlenenleri (nesne ve argümanlar) çeker, ilgili metodu çağırır ve varsa dönüş değerini yığna geri iter.

? 22. Bir sınıf yalnız yansımaya (`Class.forName`) çağrılıyorsa ve `-keep` kuralı yoksa, ProGuard/R8 bu sınıfı neden "kullanılmıyor" sanabilir? ✓

Yansımali çağrı bir dize (çalışma anında değerlendirilen metin) olarak yazılıdır; gizleyici yalnız statik çağrı grafiğini izler, dize içeriğini sınıf kullanımı saymaz. Bu yüzden sınıf küçültmede kaldırılabilir ya da adı değiştirilebilir, çalışma anında `ClassNotFoundException` / `NoSuchMethodException` oluşur.

? **23. XOR ile gizlenmiş bir dizeyi geri çözmek için saldırgana ne gerekir? Bu neden XOR gizlemeyi zayıf bir savunma yapar?** ✓

Anahtar ve çözme döngüsü, gizlenen dizeyle **aynı sınıfın içindedir**; bir tersine derleyici ikisini görüp aynı XOR işlemini otomatik tekrarlayabilir. Bu yüzden XOR gizleme kriptografik bir kilit değil, yalnız düz metin tarayan araçları (strings, javap) yavaşlatan bir engeldir.

? **24. SBOM'daki bir bileşenin bilinen bir CVE ile eşleşmesi neden otomatik olarak "etkilendik" anlamına gelmez? Bu farkı hangi belge türü bildirir?** ✓

Zafiyetli kod eşleşse bile, o fonksiyon uygulama tarafından hiç çağırılmıyor olabilir. VEX (Vulnerability Exploitability eXchange) belgeleri, bir bileşendeki bilinen zafiyetin ürünü gerçekten etkileyip etkilemediğini makine tarafından okunabilir biçimde bildirir.

? **25. Demo 1'in kötü yolunda `ad = ' OR rol='yonetici' --` girdisiyle parola alanına ne yazılırsa yazılsın giriş neden başarılı olur?** ✓

`--`, SQL'de satır sonuna kadar yorum başlatır; ayrıştırıcı bu karakterden sonraki `AND parola_ozeti = '...'` kısmını sorgunun bir parçası olarak hiç okumaz. Parola denetimi fiilen sorgudan silinmiş olur.

? **26. Bir SBOM neden yalnızca üretildiği anda doğrudur? CI hattına neden otomatik olarak eklenmelidir?** ✓

Bağımlılıklar her sürümde değişebilir (ekleme, kaldırma, sürüm yükseltme); elle üretilen bir SBOM bir sonraki değişiklikle güncelliğini yitirir ama bunu hiçbir yerde belirtmez. CI'da her derlemede otomatik üretilip bir SCA/izleme aracına gönderilirse, envanter her zaman **o anki** gerçek durumu yansıtır.

18. Kaynaklar ve ileri okuma

Ders kitabı — J. Viega, M. Messier, *Secure Programming Cookbook for C and C++*, O'Reilly, 2003:

- Tarif 3.1 (temel girdi doğrulama), 3.7 (dosya adları ve yolları doğrulama), 3.10 (siteler arası betiği önleme), 3.11 (SQL enjeksiyonunu önleme)
- Tarif 1.7–1.8 (dış programları güvenli çalıştırma)
- Tarif 12.9 (fonksiyon işaretçileri ve dolaylı çağrı — kavram), 12.11 (dizeleri gizleme)

Açık standartlar ve kaynaklar

- SEI CERT Oracle Coding Standard for Java (IDS, FIO, SER, MSC, ERR kuralları).
- OWASP: Top 10, SQL Injection Prevention Cheat Sheet, OS Command Injection Defense Cheat Sheet, XML External Entity Prevention Cheat Sheet, Deserialization Cheat Sheet; MASVS-CODE ve MASVS-RESILIENCE.
- Oracle: JEP 290 (seri durumdan çıkarma filtreleri), JEP 415 (bağlama özgü filtreler); Java Secure Coding Guidelines.
- ProGuard kılavuzu (Guardsquare) ve Android geliştirici belgeleri: "Shrink, obfuscate, and optimize your app" (R8).

- CycloneDX 1.5 ve SPDX 2.3 (ISO/IEC 5962) belirtileri; package-url (purl) belirtimi; CISA SBOM ve VEX belgeleri; SLSA çerçevesi.
- MITRE CWE: CWE-89, CWE-78, CWE-88 (argüman enjeksiyonu), CWE-22, CWE-502, CWE-611, CWE-79, CWE-798, CWE-1104 (bakımı yapılmayan üçüncü taraf bileşen kullanımı).

Sözlük

Terim	Türkçe	Kısa tanım
Managed language	Yönetilen dil	Belleği çalışma ortamının yönettiği dil (Java, C#, Python)
Injection	Enjeksiyon	Verinin bir yorumlayıcıda komut olarak işlenmesi
Prepared statement	Hazırlanmış (parametrelili) sorgu	Sorgu metni ile değerlerin ayrı gönderildiği SQL çağırısı
Path traversal	Yol geçişi	<code>../</code> gibi dizilerle izin verilen klasörün dışına çıkma
Canonicalization	Kanonikleştirme	Bir değeri tek ve kesin biçimine indirgeme
Deserialization	Seri durumdan çıkarma	Bayt dizisinden nesne oluşturma
Bytecode	Bayt kodu	Sanal makinenin çalıştırdığı ara kod
Decompiler	Tersine derleyici	Bayt kodundan kaynak kod üreten araç
Shrinking	Küçültme	Kullanılmayan kodu kaldırma
Mapping file	Eşleme dosyası	Gizlenmiş adları asıl adlara eşleyen dosya
Reflection	Yansıma	Sınıf ve metotlara çalışma anında adlarıyla erişme
SBOM	Yazılım malzeme listesi	Yazılımdaki bileşenlerin makine tarafından okunabilir listesi
purl	Paket URL'si	Bir paketi ekosistemden bağımsız tanımlayan kimlik
SCA	Yazılım bileşen analizi	Bağımlılıkları zafiyet veritabanlarıyla eşleştirme
VEX	Zafiyet sömürülebilirlik bildirimi	Bir zafiyetin ürünü etkileyip etkilemediğinin beyanı

Bir sonraki hafta

6. hafta — RASP. Bu hafta gizleme ve küçültmeyle bayt kodunun **statik** olarak okunmasını zorlaştırdık; ama saldırgan uygulamayı **çalışırken** de inceleyebilir (hata ayıklayıcı bağlamak, bellek dökümü almak, kanca takmak). 6. haftada RASP (Runtime Application Self-Protection) ile bir uygulamanın çalışma anında kendini nasıl doğruladığını, hata ayıklayıcıyı ve kancaları nasıl algıladığını göreceğiz — bu haftaki statik savunmanın üstüne inşa edilen bir sonraki katman.